



UNIVERSITÀ DEGLI STUDI DI TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Appunti di Programmazione 2

Lezioni Lab Tutor e Simulazioni d'esame

A cura di:

Mirco Negri

Versione Light Mode. Disponibile anche la [versione Dark Mode](#).

maggio 2026

Indice

1	Lezione 01: Recap delle nozioni di Programmazione 1	6
1.1	Spiegazione teorica	6
1.2	Implementazione del codice	7
1.2.1	Test sulle Variabili (<code>test_vars.cpp</code>)	7
1.2.2	Test sullo Stack Frame (<code>test_stack_frame.cpp</code>)	7
1.2.3	Test sui Puntatori e Modifiche (<code>test_puntatori.cpp</code>)	7
1.2.4	Test sull'Allocazione Dinamica (<code>test_memoria.cpp</code>)	8
1.3	Esercizi e approfondimenti dal tutorato	8
1.3.1	Esercizio base: Statistiche su Array	8
2	Lezione 02: Introduzione all'Orientamento agli Oggetti	9
2.1	Spiegazione teorica	9
2.2	Implementazione del codice	10
2.2.1	Versione 1: Struttura base e Costruttore	10
2.2.2	Versione 2 e 3: Aggiunta di comportamento (Metodi)	11
2.2.3	Uso delle Enum (<code>Direction.java</code>)	11
2.3	Esercizi e approfondimenti dal tutorato	11
2.3.1	Modellazione: Microscopio e Batterio	11
3	Lezione 03: Organizzazione del codice e pattern base	13
3.1	Spiegazione teorica	13
3.2	Implementazione del codice	14
3.2.1	Immutabilità con <code>final</code> (<code>ToolTier.java</code>)	14
3.2.2	Variabili di gioco globali e <code>static</code> (<code>GameConstants.java</code>)	15
3.2.3	Il Singleton Pattern (<code>DragonEgg.java</code>)	15
3.3	Esercizi e approfondimenti dal tutorato	16
3.3.1	Esercizio: Sistema di Settings con Singleton	16
4	Lezione 04: Incapsulamento, Invarianti e Memory Layout	17
4.1	Implementazione del codice	18
4.1.1	Protezione degli Invarianti	18
4.1.2	Passaggio dei valori per riferimento e per copia	18
4.1.3	Layout degli oggetti e memoria	19
4.2	Esercizi e approfondimenti dal tutorato	19
4.2.1	Esercizio: Test Memoria e Aliasing	20
5	Lezione 05: Ereditarietà e Polimorfismo di Sottotipo	21
5.1	Spiegazione teorica	21
5.2	Implementazione del codice	22
5.2.1	La Superclasse (<code>Entity.java</code>)	22
5.2.2	La Sottoclasse (<code>LivingEntity.java</code>)	23
5.3	Esercizi e approfondimenti dal tutorato	23
5.3.1	Gerarchia dell'Inventario (<code>Item</code> , <code>Arma</code> , <code>Pozione</code>)	23
5.3.2	Esecuzione Polimorfica e Array di Sottotipo	25
6	Lezione 06: Astrazione e Polimorfismo Ad-hoc	26
6.1	Spiegazione teorica	26

6.2	Implementazione del codice	27
6.2.1	Classi e Metodi Astratti (<code>AbstractEntity.java</code>)	27
6.2.2	Overloading vs Overriding (<code>OverloadTest.java</code>)	27
6.3	Esercizi e approfondimenti dal tutorato	28
6.3.1	Esercizio: La gerarchia Flatland	28
6.3.2	Simulazione del Mondo di Flatland	29
7	Lezione 07: Interfacce e Capacità	30
7.1	Spiegazione teorica	30
7.2	Implementazione del codice	30
7.2.1	Definizione di un'interfaccia (<code>Cancellable.java</code>)	30
7.2.2	Implementazione multipla	31
7.3	Esercizi e approfondimenti dal tutorato	31
7.3.1	Sistemi di pagamento polimorfici e Null Object Pattern	31
7.3.2	La Macchinetta (Contesto) e l'Esecuzione	33
8	Lezione 08: Composizione vs Ereditarietà e lo Strategy Pattern	34
8.1	Spiegazione teorica	34
8.2	Implementazione del codice	35
8.2.1	L'Interfaccia Strategy (<code>AttackStrategy.java</code>)	35
8.2.2	Le Strategie Concrete	35
8.2.3	La Classe di Contesto (<code>Piglin.java</code>)	36
8.2.4	Modifica del comportamento a Runtime (<code>Lecture8.main</code>)	37
8.3	Esercizi e approfondimenti dal tutorato	37
8.3.1	Esercizio: Sistema di Sconti (Strategy Pattern)	37
9	Lezione 09 (Lab 1): Modellazione, Identità degli oggetti e gerarchie	38
9.1	Spiegazione teorica e Struttura del Laboratorio	38
9.2	Implementazione pratica: Esercizio di Modellazione	39
9.3	Esercizi e approfondimenti dal tutorato: Identità e ID Univoci	40
9.3.1	Esercizio: Generatore di Matricole (<code>Studiante</code>)	40
10	Lezione 10: Il Dispatch (Static vs Dynamic Binding) e la risoluzione degli argomenti	41
10.1	Spiegazione teorica	41
10.2	Implementazione del codice	42
10.2.1	Static vs Dynamic Binding (<code>Block</code> e <code>DiamondBlock</code>)	42
10.2.2	La risoluzione degli argomenti (<code>Miner</code> e <code>Pick</code>)	43
10.3	Esercizi e approfondimenti dal tutorato: Prevedere il Binding	43
10.3.1	Esercizio: Il tranello del Dispatch	44
11	Lezione 11: V-Table, Casting, instanceof e Template Method	45
11.1	Spiegazione teorica	45
11.2	Implementazione del codice	46
11.2.1	Il problema: L'abuso di <code>instanceof</code> (<code>BadBlock.java</code>)	46
11.2.2	La soluzione: Il Template Method (<code>AbstractBlock.java</code>)	47
11.2.3	Le implementazioni concrete	47
11.3	Esercizi e approfondimenti a lezione	48
11.4	Esercizi e approfondimenti dal tutorato: Il Downcasting Sicuro	48

11.4.1	Esercizio: Verifica del Tipo Dinamico e Casting	49
11.4.2	Il Controller di Assistenza e il test di robustezza	49
12	Lezione 12 (Lab 2): Classi astratte, Metodi e Refactoring	50
12.1	Spiegazione teorica e Struttura del Laboratorio	50
12.2	Implementazione pratica: Esercizio di Refactoring	51
12.3	Esercizi e approfondimenti dal tutorato: Refactoring del Polimorfismo . . .	52
12.3.1	Esercizio: Eliminazione degli <code>instanceof</code>	52
13	Lezione 13: Gestione degli Errori ed Eccezioni	53
13.1	Spiegazione teorica	53
13.2	Implementazione del codice	54
13.2.1	Definizione di una gerarchia di Eccezioni	54
13.2.2	Sollevare e dichiarare le eccezioni (<code>CraftingTable.java</code>)	55
13.2.3	Catturare e gestire gli errori	55
13.2.4	Eccezioni e Costruttori	56
13.3	Esercizi e approfondimenti dal tutorato (Lab 3)	56
13.3.1	Esercizio: Progettazione di un sistema di Eccezioni	56
13.3.2	Gestione stratificata (Model-Controller)	57
14	Lezione 14: Modellazione e Spawner	58
14.1	Spiegazione teorica	58
14.2	Implementazione del codice	59
14.2.1	La classe Spawner (<code>Spawner.java</code>)	59
14.3	Esercizi e approfondimenti dal tutorato	59
14.3.1	Modellazione di Sistemi Gerarchici (<code>Sonda</code> e <code>SondaAvanzata</code>)	60
14.3.2	Il Gestore Centrale: Classe <code>Satellite</code>	61
15	Lezione 15: Creazione ed Evoluzione (Factory Method e State Pattern)	62
15.1	Spiegazione teorica	62
15.2	Implementazione del codice	63
15.2.1	Il Factory Method (<code>MobSpawner</code>)	63
15.2.2	Lo State Pattern (<code>Creeper</code>)	64
15.3	Esercizi e approfondimenti dal tutorato: Gestione di Stati Transizionali . .	65
15.3.1	Esercizio: La Macchinetta del Caffè (State Pattern)	66
15.3.2	Il Contesto della Macchinetta	67
16	Lezione 16 (Lab 3): Esercitazione su eccezioni, generics e pattern	68
16.1	Spiegazione teorica e Struttura del Laboratorio	68
16.2	Implementazione pratica: Classe Generica Sicura	68
16.3	Esercizi e approfondimenti dal tutorato: Il Sistema QuestLog Parametrico	69
16.3.1	Esercizio: Struttura delle Eccezioni e dell'Interfaccia Quest	69
16.3.2	Il Registro delle Missioni con i Generics (<code>QuestLog<T></code>)	71
17	Lezione 17: Uguaglianza e Ordinamento	72
17.1	Spiegazione teorica	72
17.2	Implementazione del codice	73
17.2.1	Override sicuro di <code>equals</code> e <code>hashCode</code>	73
17.2.2	Implementazione di <code>Comparable</code> (Ordinamento Naturale)	74

17.2.3	Utilizzo di Comparator (Ordinamento Alternativo a Classi Esterne)	74
17.3	Esercizi e approfondimenti dal tutorato: Uguaglianza e Ordinamento pratico	76
17.3.1	Esercizio: Uguaglianza Logica nelle Quest	76
17.3.2	Ordinamento del QuestLog	77
18	Lezione 18: Comportamenti reattivi e operazioni esterne (Observer e Visitor)	77
18.1	Spiegazione teorica	77
18.2	Implementazione del codice	79
18.2.1	L'Observer Pattern (Player e AchievementSystem)	79
18.2.2	Il Visitor Pattern (Entity e DamageVisitor)	80
18.3	Collegamento con i tutorati e sviluppi futuri	81
19	Lezione 19 (Lab 4): Esercitazione su Collections e Pattern	81
19.1	Spiegazione teorica e Struttura del Laboratorio	81
19.2	Implementazione pratica: Collections e Ordinamento	82
19.3	Esercizi e approfondimenti dal tutorato (Lab 4): Polimorfismo e Collezioni	83
19.3.1	Esercizio: Palestra, Atleti ed Enum Avanzate	84
19.3.2	Gestione Avanzata della Collezione	85
20	Lezione 20: Programmazione Funzionale e Fondamenti di JavaFX	86
20.1	Programmazione Funzionale (Interfacce Funzionali e Lambda)	86
20.1.1	Refactoring: Dalle Classi Anonime alle Lambda	87
20.1.2	Uso pratico di Predicate e Consumer	88
20.2	Fondamenti di JavaFX: Interfacce Grafiche (GUI)	89
20.2.1	I Layout Panes (Contenitori)	89
20.2.2	Costruire la finestra di base	90
20.3	Esercizi e approfondimenti dal tutorato: Unire Lambda e JavaFX	90
20.3.1	Esercizio: Filtro dinamico con Interfaccia Grafica	90
21	Lezione 21: Gestione degli Eventi in JavaFX (Event Handling)	92
21.1	Spiegazione teorica	92
21.2	Implementazione del codice	93
21.3	Collegamento con il pattern MVC	94
22	Lezione 22: Architettura del Software (Pattern Model-View-Controller)	95
22.1	Spiegazione teorica	95
22.2	Implementazione Base: multiInputsMVC	96
22.2.1	La Vista (Intercettare l'input visivo)	96
22.2.2	Il Controller (La logica di smistamento)	97
22.3	MVC Avanzato: Gestione delle Collezioni (collectionsMVC)	97
22.3.1	L'Ordinamento dei Controller (Il trucco d'esame)	98
23	Lezione 23 (Lab 5): Esercitazione su Layout grafici e implementazione MVC	99
23.1	Spiegazione teorica e Struttura del Laboratorio	99
23.2	Architettura dei Package (Il "Segreto" per l'Esame)	99
24	Lezione 24 (Lab 6): Conclusione ed Esercitazione Finale	100

24.1	Spiegazione teorica e Struttura della Simulazione	100
24.2	Caso Studio Riassuntivo: Il Sistema QuestLog	101
24.2.1	1. Il Modello: Ereditarietà e Invarianti	101
24.2.2	2. Il Modello: Gestione delle Collezioni	102
24.2.3	3. Il Controller: Il Direttore d'Orchestra	102
24.3	Conclusione e Consigli per l'Esame	103
25	Domande d'Esame: Vero o Falso	104
26	Codici d'esame	109
26.1	Output di Codice	109
26.2	Identificazione Errori	116

1 Lezione 01: Recap delle nozioni di Programmazione

1

1.1 Spiegazione teorica

Fino a questo momento, in Programmazione 1, l'approccio principale è stato il "*programming in the small*": scrittura di piccole procedure e familiarizzazione con le basi del coding. Il corso di Programmazione 2 sposta il focus sul "*programming in the large*".

Quando i progetti diventano enormi (migliaia di file e linee di codice), sorgono nuove sfide:

- **Suddivisione del lavoro:** il codice deve poter essere scritto da persone e team diversi (*divide et impera*).
- **Manutenibilità:** il codice deve essere comprensibile e modificabile anche a distanza di mesi o anni senza "rompere" funzionalità esistenti.
- **Robustezza:** prevenzione e gestione strutturata degli errori.

Per ottenere questi risultati, ci si affida a buone tecniche di programmazione, ai principi dell'Ingegneria del Software e, soprattutto, al supporto fornito dai linguaggi orientati agli oggetti (**Object-Oriented, OO**), che sono il vero fulcro di questo corso.

Per comprendere a fondo i meccanismi di Java (il linguaggio principale del corso), è essenziale costruire una **Notional Machine** corretta. Si tratta di un'astrazione mentale che ci aiuta a visualizzare cosa accade "dietro le quinte" (il runtime) quando il codice viene eseguito. A tal fine, ripassiamo quattro concetti fondamentali ereditati dal C/C++:

1. **Variabili e Scope:** Le variabili possiedono un tipo e un valore di inizializzazione. Lo *scope* ne definisce la visibilità: se dichiarata fuori dalle funzioni è *globale* (visibile ovunque), se dichiarata all'interno è *locale*. I linguaggi moderni e sicuri forzano l'inizializzazione per evitare bug subdoli.
2. **Stack Frame (Record di attivazione):** Ogni volta che si invoca una funzione, viene allocato uno "stack frame" sulla memoria Stack. Questo blocco contiene i *parametri attuali*, le *variabili locali* e l'*indirizzo di ritorno* (per sapere dove riprendere l'esecuzione al termine della funzione).
3. **Puntatori:** Un puntatore è una variabile che contiene un indirizzo di memoria. L'operatore `&` estrae l'indirizzo di una variabile, mentre l'operatore `*` (dereferenziazione) permette di accedere al valore contenuto in quell'indirizzo.
4. **Aree di Memoria:** Un programma in esecuzione è suddiviso in diverse zone:
 - **Code Section (Testo):** contiene le istruzioni compilate (spesso in sola lettura).
 - **Data Section:** contiene le variabili globali.
 - **Stack:** gestisce la memoria per le chiamate a funzione (cresce e decresce dinamicamente e automaticamente).
 - **Heap:** area dedicata all'allocazione dinamica della memoria (gestita esplicitamente dal programmatore o dal sistema).

Osservazione critica sulla gestione della memoria: Esistono tre modi principali per gestire la memoria Heap: 1. *Manuale* (come nel C/C++ tramite `new` e `delete`), che garantisce controllo ma alto rischio di memory leak. 2. *Garbage Collection* (come in Java), dove il sistema pulisce automaticamente la memoria non più raggiungibile. 3. *Ownership* (come in Rust), gestita a tempo di compilazione.

1.2 Implementazione del codice

A lezione sono stati presentati vari file C++ per dimostrare visivamente il comportamento della memoria.

1.2.1 Test sulle Variabili (`test_vars.cpp`)

```
#include <iostream>

int x = 5; // Variabile globale

void f() {
    x = x + 1; // Modifica la variabile globale
    std::cout << "x in f " << x << std::endl;
}

void g() {
    int x = 2, b = 4; // Shadowing: la x locale "nasconde" la x globale
    printf("(x,b) in g: (%d,%d)\n", x, b);
}
```

Commento: In questo snippet osserviamo la differenza tra scope globale e locale. La funzione `f()` modifica la variabile globale `x`. La funzione `g()` dichiara una sua variabile `x` locale: questo fenomeno si chiama *shadowing*. Eventuali modifiche fatte in `g()` non intaccheranno la `x` globale, proteggendo i dati esterni da alterazioni accidentali.

1.2.2 Test sullo Stack Frame (`test_stack_frame.cpp`)

```
int fact(int n) {
    if (n == 0) return 1;
    else return n * fact(n - 1);
}
```

Commento: Questa semplice funzione ricorsiva genera una pila di *stack frame*. Ad ogni chiamata `fact(n-1)`, un nuovo record di attivazione viene "impilato" sopra il precedente, con il proprio parametro `n`. Quando la condizione base `n == 0` è raggiunta, gli stack frame vengono de-allocati (fase di ritorno), risolvendo l'albero delle moltiplicazioni.

1.2.3 Test sui Puntatori e Modifiche (`test_puntatori.cpp`)

```
void incrementa_ptr(int* px) {
    *px = *px + 1;
}
```



```

void test_puntatori2() {
    int x = 1;
    incrementa_ptr(&x); // Passaggio per indirizzo
                        // (riferimento simulato in C)
    std::cout << x << std::endl; // Stampa 2
}

```

Commento: Passare una variabile "per valore" significa passarne una copia; le modifiche fatte non persistono. Passando invece l'indirizzo della variabile in memoria tramite un puntatore (&x), la funzione `incrementa_ptr` può dereferenziare il puntatore (*px) e alterare il valore originale. Questo concetto sarà vitale in Java, dove gli oggetti vengono sempre gestiti tramite riferimenti.

1.2.4 Test sull'Allocazione Dinamica (`test_memoria.cpp`)

```

void test_mem_dyn() {
    int* px;
    px = new int;      // Allocazione dinamica sulla Heap
    *px = 3;
    std::cout << *px << std::endl;
    delete(px);       // De-allocazione manuale necessaria nel C++
}

```

Commento: La variabile puntatore `px` risiede nello *Stack* (poiché è locale alla funzione), ma punta a un'area di memoria creata nella *Heap* tramite la keyword `new`. Nel C++, il programmatore è responsabile di distruggere l'allocazione tramite `delete(px)`, altrimenti si incorre in memory leak. Nel corso vedremo come Java ci sollevi da questa responsabilità grazie al suo *Garbage Collector*.

1.3 Esercizi e approfondimenti dal tutorato

Il **Tutorato 1** inizia con una transizione guidata dal C++ verso Java. Prima di tuffarsi nella programmazione ad oggetti vera e propria, vengono affrontati classici algoritmi imperativi per prendere confidenza con la sintassi Java, l'allocazione degli array e l'uso dello `Scanner` per l'input utente.

1.3.1 Esercizio base: Statistiche su Array

Di seguito l'implementazione per il calcolo di media e mediana su un array di interi. Un dettaglio architetturale vitale che viene introdotto qui è l'uso di `array.clone()` nel calcolo della mediana: poiché in Java gli array sono oggetti passati per riferimento, usare `Arrays.sort(array)` ordinerebbe l'array originale anche nel `main`, causando un *side-effect* distruttivo non desiderato.

```

import java.util.Arrays;
import java.util.Scanner;

public class MediaMedianaModa {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

```

```

        System.out.print("Inserire la dimensione dell'array (n): ");
        int n = sc.nextInt();
        int[] values = new int[n];

        System.out.println("Inserire i valori:");
        for (int i = 0; i < n; i++) {
            values[i] = sc.nextInt();
        }

        System.out.println("Media: " + calcolaMedia(values));
        System.out.println("Mediana: " + calcolaMediana(values));
    }

    public static double calcolaMedia(int[] array) {
        if (array.length == 0) return 0;
        double somma = 0;
        for (int valore : array) {
            somma += valore; // Costrutto for-each nativo di Java
        }
        return somma / array.length;
    }

    public static double calcolaMediana(int[] array) {
        if (array.length == 0) return 0;

        // Copiamo l'array per evitare di modificare quello originale
        int[] ordinato = array.clone();
        Arrays.sort(ordinato);

        int n = ordinato.length;
        if (n % 2 == 0) {
            // Se la lunghezza è pari, si fa la media dei due elementi centrali
            return (ordinato[n / 2 - 1] + ordinato[n / 2]) / 2.0;
        } else {
            // Se dispari, si prende l'elemento centrale
            return ordinato[n / 2];
        }
    }
}

```

2 Lezione 02: Introduzione all'Orientamento agli Oggetti

2.1 Spiegazione teorica

In questa lezione passiamo dal paradigma procedurale a quello Orientato agli Oggetti (OOP). Se nella programmazione procedurale il focus è sulle funzioni che manipolano dati,

nella OOP il focus è sulle "entità" che raggruppano sia i dati che il comportamento.

- **Classe:** È il "progetto" o "stampo" (blueprint). Definisce quali dati (**campi** o attributi) e quali azioni (**metodi**) avranno gli oggetti di quel tipo.
- **Oggetto (Istanza):** È l'entità concreta creata in memoria a partire dalla classe. Ogni oggetto ha il proprio stato (valori dei campi).
- **Costruttore:** Un metodo speciale chiamato quando si crea un oggetto con la keyword **new**. Serve a inizializzare lo stato interno. In Java, se non ne scriviamo uno, ne esiste uno "di default" senza parametri.
- **Il riferimento `this`:** È una parola chiave che indica "questo oggetto specifico". Si usa spesso nel costruttore per distinguere i parametri della funzione dai campi della classe (es. `this.x = x`).

Tipi Enumerativi (Enum): Spesso abbiamo bisogno di variabili che possono assumere solo un set ristretto di valori costanti (es. i giorni della settimana o i punti cardinali). In Java, gli `enum` sono tipi di dato che rendono il codice più leggibile e "type-safe" rispetto all'uso di semplici interi o stringhe.

2.2 Implementazione del codice

Vediamo l'evoluzione della classe `TNT` usata a lezione per simulare un blocco del gioco Minecraft.

2.2.1 Versione 1: Struttura base e Costruttore

In questa fase definiamo solo lo stato (posizione `x`, `y`, `z`).

```
public class TNT {
    public int x, y, z;

    public TNT(int x, int y, int z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }
}
```

Commento: I campi sono `public` (accessibili da chiunque, pratica che vedremo essere sconsigliata) e il costruttore usa `this` per assegnare i valori ricevuti ai campi dell'oggetto.

2.2.2 Versione 2 e 3: Aggiunta di comportamento (Metodi)

Aggiungiamo un metodo per far "esplodere" il blocco e gestire lo stato `primed`.

```
public class TNT {
    public int x, y, z;
    public boolean isPrimed;

    public TNT(int x, int y, int z) {
        this.x = x;
        this.y = y;
        this.z = z;
        this.isPrimed = false;
    }

    public void explode() {
        if (isPrimed) {
            System.out.println("BOOM at " + x + ", " + y + ", " + z);
        } else {
            System.out.println("TNT is not primed yet.");
        }
    }
}
```

Commento: Qui notiamo come il metodo `explode()` utilizzi i dati interni dell'oggetto (`x`, `y`, `z`, `isPrimed`) per decidere cosa fare. Questo è il cuore della OOP: l'oggetto "sa" come comportarsi in base al suo stato.

2.2.3 Uso delle Enum (`Direction.java`)

```
public enum Direction {
    NORTH, SOUTH, EAST, WEST, UP, DOWN
}
```

Commento: Invece di usare numeri (0, 1, 2...) che potrebbero confondere, usiamo nomi chiari. Java garantisce che una variabile di tipo `Direction` possa contenere solo uno di questi sei valori.

2.3 Esercizi e approfondimenti dal tutorato

Il **Tutorato 1** introduce la modellazione di sistemi complessi tramite classi, `enum` e l'uso corretto dei costruttori. Un esempio fondamentale è l'esercizio **Microscopio e Batterio**, che insegna a definire il dominio di un problema tramite i sostantivi presenti nel testo.

2.3.1 Modellazione: Microscopio e Batterio

L'esercizio richiede di modellare i batteri in base alla loro morfologia e i microscopi in base alla tecnologia utilizzata, garantendo che gli oggetti non possano esistere in stati non validi (es. un batterio senza morfologia).

```
// 1. Enum per evitare "magic numbers" o stringhe fragili
public enum Morfologia { BACILLO, COCCO, SPIRILLO }
public enum TipoMicroscopio { OTTICO, ELETTRONICO }

// 2. Classe Batterio: l'immutabilità è garantita da 'final'
public class Batterio {
    private final String nome;
    private final Morfologia tipo;

    public Batterio(String nome, Morfologia tipo) {
        this.nome = nome;
        this.tipo = tipo;
    }
}

// 3. Classe Microscopio: gestione dello stato
public class Microscopio {
    private TipoMicroscopio tecnologia;
    private int ingrandimento;

    public Microscopio(TipoMicroscopio tecnologia, int ingrandimento) {
        this.tecnologia = tecnologia;
        this.ingrandimento = ingrandimento;
    }

    public void osserva(Batterio b) {
        System.out.println("Osservo il batterio con tecnologia: " + tecnologia);
    }
}
```

Analisi didattica:

- **Uso delle Enum:** Invece di usare stringhe come "Bacillo", l'uso di `Morfologia.BACILLO` impedisce errori di battitura e rende il codice "type-safe".
- **Immutabilità:** Il campo `nome` del batterio è `final` perché l'identità del batterio non deve cambiare dopo la sua creazione, prevenendo bug di mutazione indesiderata dello stato.
- **Modularità:** La separazione tra `Batterio` e `Microscopio` permette di espandere le funzionalità (es. aggiungere nuovi tipi di microscopi) senza dover modificare le classi esistenti, aderendo ai principi di progettazione dell'OOP.

3 Lezione 03: Organizzazione del codice e pattern base

3.1 Spiegazione teorica

Man mano che un progetto cresce, scrivere tutto il codice in un unico file diventa insostenibile. In questa lezione esploriamo i meccanismi offerti da Java per organizzare, proteggere e condividere le funzionalità: *packages*, modificatori di visibilità, parole chiave per l'immutabilità e la globalità, fino ad arrivare a un primo *Design Pattern*.

- **Packages:** Sono i "namespace" di Java, corrispondenti fisicamente alle cartelle sul disco. Risolvono i conflitti di nome (es. avere due classi TNT in pacchetti diversi) e organizzano logicamente il progetto. Per usare classi di altri pacchetti si usa la direttiva `import`.
- **Visibilità e Incapsulamento (Information Hiding):** Nascondere i dettagli implementativi all'interno di una classe è fondamentale per evitare alterazioni esterne indesiderate (es. cambiare la vita di un giocatore ignorando l'armatura).
 - `public`: visibile ovunque.
 - `private`: visibile solo all'interno della classe stessa.
 - `protected`: visibile all'interno dello stesso package e alle sottoclassi.
 - *Package-private* (default, nessuna keyword): visibile solo all'interno dello stesso package.

Per leggere o scrivere in modo controllato i campi `private`, si definiscono metodi pubblici chiamati **getter** e **setter**.

- **Immutabilità (`final`):** Applicata a una variabile o a un campo, la keyword `final` lo rende una *costante*. Una volta assegnato (o al momento della dichiarazione o nel costruttore), il valore non può più cambiare. Questo previene bug legati alla mutazione inattesa dello stato.
- **Globalità (`static`):** Un campo o metodo `static` appartiene alla *Classe* e non alla singola istanza (oggetto). Di conseguenza:
 - Esiste una sola copia in memoria per tutta l'esecuzione.
 - Vi si accede usando il nome della classe (es. `Classe.metodo()`).
 - Non si può usare `this` all'interno di metodi `static`.
 - Utile per definire costanti globali (es. `public static final int MAX_STACK = 64`).
- **Design Pattern - Singleton:** I Design Pattern sono soluzioni standard e testate a problemi ricorrenti. Il **Singleton** garantisce che esista *una sola istanza* di una classe nell'intero programma (utile ad esempio per i Settings o oggetti unici del gioco, come il Dragon Egg).

3.2 Implementazione del codice

```
package lecture03.ackages.entities;

public class Player {
    public String username;           // Leggibile e modificabile da tutti
    private int health = 20;          // Nascosto, accessibile solo
                                      // ai metodi interni

    private boolean isPoisoned = false;

    // Setter per controllare in modo sicuro lo stato
    public void setPoisoned(boolean p) {
        this.isPoisoned = p;
    }

    // Getter per leggere lo stato in sola lettura
    public String getUsername() {
        return this.username;
    }

    public void damage(int amount) {
        this.health = this.health - amount;
        if (this.health <= 0) {
            System.out.println("Player died!");
        }
    }
}
```

Commento: Il campo `health` è `private`. Nessun'altra classe può fare `player.health = 0`, "barando" e aggirando la logica di danno del gioco. L'unico modo per sottrarre vita è tramite il metodo pubblico `damage(int amount)`. Questo è l'**Information Hiding**.

3.2.1 Immutabilità con `final` (ToolTier.java)

```
package lecture03.final_mod;

public enum ToolTier {
    WOOD(59, 2.0f), DIAMOND(1561, 8.0f);

    private final int maxUses;
    private final float efficiency;

    ToolTier(int maxUses, float efficiency) {
        this.maxUses = maxUses;
        this.efficiency = efficiency;
    }

    public float getEfficiency() { return this.efficiency; }
}
```

Commento: La velocità (*efficiency*) del diamante deve essere sempre 8.0f. Rendendola *final*, garantiamo a livello di compilatore che non possa essere alterata accidentalmente a runtime.

3.2.2 Variabili di gioco globali e static (*GameConstants.java*)

```
package lecture03.static_globals;

public class GameConstants {
    // Un valore condiviso da tutto il server
    public static final int MAX_STACK_SIZE = 64;

    // Metodo Utility statico: non ha bisogno dello stato di un oggetto
    public static void printMOTD() {
        System.out.println(">> Welcome to the Minecraft Course!");
    }
}
```

Commento: *MAX_STACK_SIZE* è *static*, quindi è condivisa; non ne viene creata una copia in memoria per ogni blocco o inventario esistente. Poiché è anche *final*, nessuno può modificarne il valore. Si richiamerà con *GameConstants.MAX_STACK_SIZE*.

3.2.3 Il Singleton Pattern (*DragonEgg.java*)

```
package lecture03.static_globals;

public class DragonEgg {
    // 1. Unica istanza statica, creata al momento del caricamento
    private static DragonEgg THE_INSTANCE = new DragonEgg();

    // 2. Costruttore privato (nessuno può fare 'new DragonEgg()' fuori da qui)
    private DragonEgg() {}

    // 3. Getter pubblico e statico per accedere all'unica istanza
    public static DragonEgg getInstance() {
        return THE_INSTANCE;
    }

    public void teleport() {
        System.out.println(">> Vwoop! The Dragon Egg teleported away.");
    }
}
```

Commento: Questo pattern ruota attorno all'uso combinato di *private* e *static*. Rendendo il costruttore privato, blocchiamo la creazione di nuovi oggetti. Offriamo invece l'accesso alla singola istanza globale tramite il metodo *getInstance()*. Questo assicura, ad esempio, che non si duplichino accidentalmente le risorse o gli asset unici del programma.

3.3 Esercizi e approfondimenti dal tutorato

Il **Tutorato 2** approfondisce la modellazione di sistemi composti. L'obiettivo è passare dalla classe singola alla gestione di *oggetti che contengono altri oggetti* (Composizione), garantendo che lo stato del sistema rimanga sempre coerente.

3.3.1 Esercizio: Sistema di Settings con Singleton

Spesso in un'applicazione serve una classe che contenga le preferenze di sistema (es. volume, tema, risoluzione). Questa classe deve essere accessibile ovunque e non deve essere duplicata: è qui che interviene il pattern **Singleton**.

```
public class Settings {
    // 1. Istanza unica statica (Eager initialization)
    private static final Settings INSTANCE = new Settings();

    private int volume;
    private String theme;

    // 2. Costruttore privato: nessuno può fare 'new Settings()'
    private Settings() {
        this.volume = 50;
        this.theme = "DARK";
    }

    // 3. Metodo statico per accedere all'unica istanza
    public static Settings getInstance() {
        return INSTANCE;
    }

    public void setVolume(int v) { this.volume = v; }
    public int getVolume() { return volume; }
}
```

Analisi didattica:

- **Accesso Globale:** Ora, ovunque nel codice (es. nel menu o nelle opzioni), posso scrivere `Settings.getInstance().getVolume()` senza dover passare l'oggetto di mano in mano tra i metodi.
- **Controllo dell'istanza:** Rendendo il costruttore `private`, impediamo che parti diverse del software creino copie divergenti delle impostazioni.
- **Composizione:** L'esercizio del tutorato estende questo concetto creando classi come `AudioSettings` o `GraphicsSettings` che vengono iniettate nel `Settings` principale, mostrando come i sistemi software reali siano costruiti per aggregazione di componenti specializzati.

4 Lezione 04: Incapsulamento, Invarianti e Memory Layout

In questa lezione si approfondiscono le conseguenze dell'incapsulamento, si introduce il concetto di **invariante** e si analizza in dettaglio come Java gestisce la memoria "dietro le quinte", in particolare il passaggio dei parametri e il layout di classi e oggetti (V-Table).

- **Incapsulamento e Invarianti:** L'incapsulamento non serve solo a nascondere le informazioni, ma è lo strumento principale per garantire gli *invarianti*. Un invariante è una proprietà o una condizione di un oggetto che deve rimanere **sempre vera** durante tutta la sua esistenza. Ad esempio: la vita di un giocatore non può essere negativa, oppure il fuso di una TNT non può essere modificato con valori arbitrari (es. -100). Per far rispettare questi invarianti, si rendono i campi **private** e si gestisce la logica di mutazione unicamente attraverso metodi di classe controllati.
- **Passaggio dei valori (Value vs Reference):** Quando si passa una variabile a un metodo, il comportamento dipende dal tipo di dato:
 - **Tipi Primitivi** (`int`, `double`, `boolean`...): Vengono allocati sullo *Stack* e passati per **Copia** (Passaggio per valore). Le modifiche fatte dal metodo chiamato non si riflettono sulla variabile originale.
 - **Oggetti e Array:** Vengono allocati nella *Heap*. La variabile nello *Stack* contiene in realtà un **Riferimento** (un puntatore) all'oggetto. Quando si passa un oggetto a un metodo, si copia il *riferimento*. Di conseguenza, le modifiche fatte allo stato dell'oggetto all'interno del metodo si rifletteranno anche all'esterno, poiché entrambe le variabili puntano alla stessa area di memoria.
- **Memory Layout e V-Table:** Quando il programma viene caricato, il codice dei metodi (**static** e non) va nella *Code Section*, mentre i campi **static** vanno in un'area *Read-Only*. Ma come fa un oggetto a sapere quali metodi può eseguire? Tramite la **V-Table** (Virtual Table).
 - Ogni classe possiede una V-Table, ovvero un array di puntatori che indicano l'indirizzo in memoria delle implementazioni dei suoi metodi.
 - Ogni oggetto istanziato nella *Heap* ha una struttura precisa: la prima parola (word) di memoria è un puntatore nascosto alla V-Table della sua classe. Le parole successive contengono i valori dei suoi campi (stato).
 - Quando si invoca un metodo su un oggetto (`p.damage()`), Java segue il puntatore alla V-Table, cerca l'offset del metodo e ne esegue il codice corrispondente.

4.1 Implementazione del codice

4.1.1 Protezione degli Invarianti

```
public static void invariants() {
    TNT standardTNT = new TNT();

    // Se fuseLength fosse public, potremmo violare l'invariante del gioco:
    // standardTNT.fuseLength = -100; // ORA E' UN ERRORE DI COMPILAZIONE!

    // Utilizzo corretto e sicuro tramite i metodi previsti:
    standardTNT.ignite();
    standardTNT.tick();
}
```

Commento: Rendendo `fuseLength` e `isIgnited` privati, costringiamo chi usa la classe `TNT` a invocare i metodi `ignite()` e `tick()`, che al loro interno contengono i controlli di sicurezza (es. non far scendere il fuso sotto lo zero, non esplodere se non innescata).

4.1.2 Passaggio dei valori per riferimento e per copia

```
private static void valuePassingExample(){
    int x = 0;
    helper(x);
    // x rimane 0. È stato passato il "valore" 0 copiandolo nel nuovo stack frame.

    TNT t = new TNT();
    reference(t);
    // Se "reference()" modifica lo stato di 't', lo vedremo anche qui.
    // Viene passato l'indirizzo di memoria dell'oggetto.

    TNT[] arrT = new TNT[5];
    // In questo momento abbiamo creato solo "lo spazio" per l'array.
    // Le celle contengono 'null' (il valore di default per i riferimenti).

    for (int i = 0 ; i < 5 ; i++){
        arrT[i] = new TNT(); // Ora creiamo concretamente gli oggetti nella Heap
    }
}
```

Commento: L'errore più comune nei linguaggi ad oggetti è la cosiddetta eccezione di `NullPointerException`. Questo avviene quando si cerca di invocare un metodo o leggere un campo su una variabile riferimento che non punta ad alcun oggetto allocato (come le celle di `arrT` prima del ciclo `for`).

4.1.3 Layout degli oggetti e memoria

```
private static void layoutTest(){
    Player p1 = new Player();
    Player p2 = new Player();

    p2.username = "steve";
    p1.setPoisoned(true);
}
```

Commento (Analisi del Runtime): Quando viene eseguito questo codice: 1. Nello *Stack* del *main* vengono create due variabili, *p1* e *p2*. 2. Nella *Heap* vengono allocati due blocchi di memoria separati. 3. Il layout di *p1* nella memoria *Heap* sarà: - Indirizzo: 0x00A000 (Puntatore alla V-Table di *Player*) - Indirizzo: 0x00A004 (*null*, valore di default di *username*) - Indirizzo: 0x00A008 (20, valore di default di *health*) - Indirizzo: 0x00A00C (*true*, perché modificato da *setPoisoned*)

4.2 Esercizi e approfondimenti dal tutorato

Nel **Tutorato 2**, l'esercizio *TestMemoria.java* esplora i concetti di allocazione e passaggio dei parametri, chiedendo di prevedere il comportamento delle variabili. Si pone l'enfasi sul fenomeno dell'**Aliasing** (più variabili che puntano allo stesso oggetto) e sulla differenza tra l'operatore *==* e l'uguaglianza logica.

4.2.1 Esercizio: Test Memoria e Aliasing

```
public class Auto {
    private String modello;

    public Auto(String modello) {
        this.modello = modello;
    }

    public void setModello(String m) {
        this.modello = m;
    }

    public String getModello() {
        return this.modello;
    }
}

public class TestMemoria {
    public static void main(String[] args) {
        Auto a = new Auto("SUV");
        Auto b = a;           // ALIASING: copia del riferimento
        Auto c = new Auto("SUV"); // Nuovo oggetto allocato nella Heap

        // Modifico l'oggetto tramite la variabile 'b'
        b.setModello("CityCar");

        // Previsione dell'output:
        System.out.println(a.getModello()); // Stampa "CityCar"

        System.out.println(a == b);         // Stampa: true
        System.out.println(a == c);         // Stampa: false
    }
}
```

Analisi didattica:

- **Lo Stack e la Heap:** Quando scriviamo `Auto b = a;`, non stiamo creando una seconda automobile. `a` e `b` sono due variabili distinte sullo *Stack*, ma contengono l'esatto stesso indirizzo che punta all'unico oggetto nella *Heap*.
- **Side-Effects (Effetti Collaterali):** Di conseguenza, modificando il modello tramite `b`, la modifica si riflette istantaneamente anche interrogando l'oggetto tramite `a`. Questo fenomeno è alla base di moltissimi bug ed è il motivo principale per cui l'incapsulamento (`private`) e l'immutabilità (`final`) sono regole d'oro nell'architettura software.
- **L'operatore ==:** Verifica esclusivamente l'identità (stesso indirizzo di memoria). `a` e `c`, pur avendo originariamente lo stesso stato interno ("SUV"), risiedono in due aree di memoria differenti, pertanto `a == c` risulta `false`.

5 Lezione 05: Ereditarietà e Polimorfismo di Sottotipo

5.1 Spiegazione teorica

L'ereditarietà è uno dei pilastri fondamentali della programmazione orientata agli oggetti. Permette di definire una nuova classe (sottoclasse) a partire da una classe esistente (superclasse), ereditandone campi e metodi.

- **Relazione "is-a":** Una sottoclasse è *un* tipo specifico della superclasse (es. un **Cane** è un **Animale**). Questo permette il riutilizzo del codice e la creazione di gerarchie logiche.
- **Keyword `extends`:** In Java si usa per stabilire il legame di ereditarietà. Una classe può estendere **una sola** altra classe (ereditarietà singola).
- **Polimorfismo di Sottotipo:** È la capacità di un riferimento di tipo superclasse di puntare a un oggetto di una sottoclasse. Ad esempio, una variabile di tipo **Entity** può contenere un oggetto **Player**. Questo è fondamentale per scrivere codice generico che funzioni con intere gerarchie.
- **Costruttori e `super()`:** I costruttori non vengono ereditati. La sottoclasse deve chiamare il costruttore della superclasse tramite la keyword **`super(...)`** come prima istruzione del proprio costruttore. Se non specificato, Java inserisce automaticamente una chiamata a **`super()`** senza parametri.
- **La classe `Object`:** In Java, ogni classe che non specifica un genitore estende implicitamente **`java.lang.Object`**. Essa è la radice di ogni gerarchia e fornisce metodi universali come **`toString()`**, **`equals()`** e **`hashCode()`**.
- **Diamond Problem:** Java non permette l'ereditarietà multipla di classi (estendere due classi contemporaneamente) per evitare ambiguità: se due genitori hanno un metodo con la stessa firma ma implementazione diversa, la sottoclasse non saprebbe quale ereditare.

5.2 Implementazione del codice

A lezione è stata presentata la gerarchia delle entità, simile a quella di un motore di gioco come Minecraft.

5.2.1 La Superclasse (Entity.java)

```
public class Entity {
    protected double x, y, z; // Visibile alle sottoclassi

    public Entity(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }

    public void move(double dx, double dy, double dz) {
        this.x += dx;
        this.y += dy;
        this.z += dz;
    }
}
```

Commento: L'uso del modificatore `protected` permette alle classi figlie di accedere direttamente alle coordinate, mantenendole però nascoste al resto del programma (incapsulamento parziale).

5.2.2 La Sottoclasse (LivingEntity.java)

```
public class LivingEntity extends Entity {
    private int health;

    public LivingEntity(double x, double y, double z, int health) {
        super(x, y, z); // Chiamata obbligatoria al costruttore del genitore
        this.health = health;
    }

    public void takeDamage(int amount) {
        this.health -= amount;
        if (this.health <= 0) {
            System.out.println("Entity is dead.");
        }
    }
}
```

Commento: LivingEntity aggiunge lo stato `health` e il comportamento `takeDamage`, ma possiede anche le coordinate e il metodo `move` ereditati da `Entity`.

5.3 Esercizi e approfondimenti dal tutorato

Nel **Tutorato 2**, l'ereditarietà e il polimorfismo di sottotipo vengono applicati alla gestione di un inventario polimorfico tramite una gerarchia di oggetti (`Item`). L'obiettivo è comprendere come trattare entità diverse sotto un'unica interfaccia comune, sfruttando il riutilizzo del codice.

5.3.1 Gerarchia dell'Inventario (`Item`, `Arma`, `Pozione`)

Di seguito viene mostrata la struttura delle tre classi del dominio e un ciclo d'esecuzione che illustra il comportamento del polimorfismo di sottotipo.


```

// 1. Superclasse generica
public class Item {
    private final String nome;

    public Item(String nome) {
        this.nome = nome;
    }

    public String getNome() { return nome; }

    public void usa() {
        System.out.println("Usi l'oggetto generico: " + nome);
    }
}

// 2. Sottoclasse specializzata: Arma
public class Arma extends Item {
    private final int danno;

    public Arma(String nome, int danno) {
        super(nome); // Inizializza obbligatoriamente la parte "Item"
        this.danno = danno;
    }

    @Override
    public void usa() {
        System.out.println("Sguaini"+getNome()+"infliggendo"+danno+"danni!");
    }
}

// 3. Sottoclasse specializzata: Pozione
public class Pozione extends Item {
    private final int cura;

    public Pozione(String nome, int cura) {
        super(nome);
        this.cura = cura;
    }

    @Override
    public void usa() {
        System.out.println("Bevi " + getNome() + " recuperando " + cura + " HP!");
    }
}

```

5.3.2 Esecuzione Polimorfica e Array di Sottotipo

Nel `main` del tutorato viene creato un array polimorfico. Notare come sia possibile inserire elementi di tipo `Arma` o `Pozione` dentro un contenitore di tipo `Item`.

```
public class TestInventario {
    public static void main(String[] args) {
        // Un riferimento di tipo superclasse può puntare a oggetti sottoclasse
        Item spada = new Arma("Excalibur", 50);
        Item pozione = new Pozione("Cura Maggiore", 20);

        // Creazione di un array polimorfico
        Item[] inventario = { spada, pozione };

        for (Item i : inventario) {
            // A tempo di compilazione Java vede solo il tipo statico (Item)
            // A runtime eseguirà il metodo dell'oggetto reale (Tipo Dinamico)
            i.usa();
        }
    }
}
```

Analisi didattica:

- **Relazione Is-A:** Poiché un' `Arma` è *un* `Item`, l'assegnazione iniziale è del tutto lecita per il polimorfismo di sottotipo.
- **Il limite del Tipo Statico:** Agendo tramite un riferimento generico (la variabile `i` nel ciclo), il compilatore ci permette di invocare solo i metodi definiti in `Item`. Se `Arma` avesse un metodo specifico (es. `getDanno()`), non potremmo chiamarlo direttamente su `i` senza un'operazione di casting sicuro (**downcasting**), che verrà approfondita nelle lezioni successive.
- **Anticipazione del Dynamic Dispatch:** Il ciclo `for` non ha bisogno di verificare se l'oggetto sia un'arma o una pozione tramite costrutti condizionali. Invoca semplicemente `i.usa()` e Java si occupa autonomamente di smistare la chiamata all'implementazione corretta a runtime.

6 Lezione 06: Astrazione e Polimorfismo Ad-hoc

6.1 Spiegazione teorica

L'astrazione è il processo di isolamento delle proprietà essenziali di un'entità, tralasciando i dettagli implementativi che non sono comuni a tutte le sue varianti.

- **Classi Astratte (`abstract class`):** Una classe dichiarata come astratta non può essere istanziata direttamente (non si può fare `new`). Serve come modello parziale per le sottoclassi. Può contenere sia metodi implementati che metodi astratti.
- **Metodi Astratti (`abstract method`):** Sono metodi dichiarati senza un corpo (senza implementazione). Rappresentano una "promessa": ogni sottoclasse concreta **deve** fornire un'implementazione per quel metodo.
- **Polimorfismo Ad-hoc:** Si riferisce alla capacità di un metodo di comportarsi in modo diverso in base ai tipi dei parametri o alla classe che lo implementa. Si divide in:
 - **Overloading (Sovraccarico):** Più metodi nella stessa classe con lo **stesso nome** ma **parametri diversi** (per tipo o numero). È risolto a tempo di compilazione (*static polymorphism*).
 - **Overriding (Sovrascrittura):** Una sottoclasse fornisce una nuova implementazione di un metodo già definito nella superclasse (stessa firma). È risolto a runtime (*dynamic polymorphism*).

6.2 Implementazione del codice

6.2.1 Classi e Metodi Astratti (AbstractEntity.java)

```
public abstract class AbstractEntity {
    protected int x, y;

    public AbstractEntity(int x, int y) {
        this.x = x;
        this.y = y;
    }

    // Metodo concreto: tutte le entità si muovono allo stesso modo
    public void moveTo(int newX, int newY) {
        this.x = newX;
        this.y = newY;
    }

    // Metodo astratto: ogni entità ha un modo diverso di "esistere" o agire
    public abstract void update();
}
```

Commento: AbstractEntity definisce la struttura comune (coordinate) e un comportamento condiviso (moveTo), ma delega il comportamento specifico (update) alle classi figlie.

6.2.2 Overloading vs Overriding (OverloadTest.java)

```
public class Calculator {
    // OVERLOADING: stesso nome, parametri diversi
    public int sum(int a, int b) { return a + b; }
    public double sum(double a, double b) { return a + b; }
}

public class SpecialCalculator extends Calculator {
    // OVERRIDING: stessa firma della superclasse, comportamento diverso
    @Override
    public int sum(int a, int b) {
        System.out.println("Using special sum!");
        return a + b + 1;
    }
}
```

Commento: L'overloading permette di gestire tipi diversi con lo stesso nome logico. L'overriding, segnalato dall'annotazione @Override, permette di specializzare un comportamento ereditato.

6.3 Esercizi e approfondimenti dal tutorato

Nel **Tutorato 3**, il concetto di astrazione viene applicato all'esercizio **Flatland**, ispirato al celebre romanzo fantascientifico. L'obiettivo è modellare gli abitanti di un mondo geometrico che possiedono caratteristiche spaziali differenti (es. 1D e 2D) ma che devono rispondere a un'interfaccia logica comune per il calcolo della propria estensione.

6.3.1 Esercizio: La gerarchia Flatland

L'uso di una classe astratta impedisce l'istanza di un "abitante generico" (che non avrebbe senso geometrico) e impone un contratto rigoroso a tutte le sottoclassi concrete.

```
// 1. Superclasse astratta che definisce il concetto di abitante
public abstract class Flatlander {
    private final String nome;

    public Flatlander(String nome) {
        this.nome = nome;
    }

    public String getNome() { return nome; }

    // Metodo astratto: ogni abitante ha un modo totalmente diverso
    // di calcolare la propria "misura" spaziale
    public abstract double getMeasure();

    @Override
    public String toString() {
        return nome + " [Misura: " + getMeasure() + "]";
    }
}

// 2. Sottoclasse 1D: Linelander (possiede solo una lunghezza)
public class Linelander extends Flatlander {
    private final double length;

    public Linelander(String nome, double length) {
        super(nome);
        this.length = length;
    }

    @Override
    public double getMeasure() {
        return this.length;
    }
}
```

```
// 3. Sottoclasse 2D: Squarelander (possiede un lato, la misura è l'area)
public class Squarelander extends Flatlander {
    private final double side;

    public Squarelander(String nome, double side) {
        super(nome);
        this.side = side;
    }

    @Override
    public double getMeasure() {
        return this.side * this.side; // L'area rappresenta la misura 2D
    }
}
```

6.3.2 Simulazione del Mondo di Flatland

Nel `main` del tutorato viene creata una popolazione eterogenea per dimostrare come sia possibile manipolare gli oggetti sfruttando esclusivamente la loro radice comune astratta.

```
import java.util.ArrayList;
import java.util.List;

public class FlatlandSimulation {
    public static void main(String[] args) {
        List<Flatlander> abitanti = new ArrayList<>();

        abitanti.add(new Linelander("Linea_A", 12.5));
        abitanti.add(new Squarelander("Quadrato_B", 4.0));

        System.out.println("--- Censimento di Flatland ---");
        for (Flatlander f : abitanti) {
            // Sfrutta il Dynamic Dispatch: invoca l'override specifico a runtime
            System.out.println(f);
        }
    }
}
```

Analisi critica:

- **Inibizione dell'istanza:** Dichiarare `Flatlander` come `abstract` impedisce al programmatore di fare `new Flatlander("Errore")`, proteggendo la coerenza semantica del modello dati.
- **Obbligo di Overriding:** Il compilatore garantisce che qualsiasi nuova forma aggiunta al sistema (es. un ipotetico `Circlelander`) implementi obbligatoriamente il metodo `getMeasure()`, evitando fallimenti o comportamenti indefiniti.
- **Estensibilità dell'Architettura:** Questa struttura rispetta appieno il principio *Open/Closed*: se l'applicazione venisse estesa con decine di nuove forme geometriche, il ciclo di censimento nel simulatore rimarrebbe perfettamente invariato e funzionante.

7 Lezione 07: Interfacce e Capacità

7.1 Spiegazione teorica

Se le classi astratte rappresentano "cosa un oggetto è" (un'entità, un animale), le interfacce rappresentano "cosa un oggetto sa fare" (una capacità). In Java, un'interfaccia è un contratto puramente formale.

- **Definizione:** Un'interfaccia può contenere solo firme di metodi (di default pubblici e astratti) e costanti (`public static final`). Non può avere campi di istanza (stato) né costruttori.
- **Ereditarietà Multipla di Tipo:** A differenza delle classi, una classe può implementare infinite interfacce (`implements A, B, C...`). Questo permette a un oggetto di assumere molteplici "ruoli" polimorfici.
- **Metodi default (Java 8+):** Introdotti per permettere l'evoluzione delle interfacce senza rompere il codice esistente. Consentono di scrivere un'implementazione standard all'interno dell'interfaccia stessa. Se una classe eredita due metodi `default` identici da due interfacce diverse, Java obbliga il programmatore a fare l'override per risolvere l'ambiguità.
- **Interfacce vs Classi Astratte:**
 - *Classe Astratta:* Si usa per condividere codice e stato tra classi strettamente correlate (relazione forte "is-a").
 - *Interfaccia:* Si usa per definire un protocollo di comunicazione o una capacità comune a classi anche molto diverse tra loro (relazione debole "can-do").

7.2 Implementazione del codice

Vediamo come definire un'interfaccia per oggetti che possono essere "annullati" (come un'esplosione o un comando).

7.2.1 Definizione di un'interfaccia (`Cancellable.java`)

```
public interface Cancellable {  
    // Un campo in un'interfaccia è implicitamente public static final  
    int MAX_DELAY = 100;  
  
    // Un metodo è implicitamente public e abstract  
    void cancel();  
  
    // Metodo default: fornisce un'implementazione base  
    default void logCancel() {  
        System.out.println("Operation was cancelled.");  
    }  
}
```

7.2.2 Implementazione multipla

```
public class ExplodingTNT extends TNT implements Cancellable, Runnable {
    @Override
    public void cancel() {
        this.isPrimed = false;
        System.out.println("Explosion averted.");
    }

    @Override
    public void run() {
        this.explode();
    }
}
```

Commento: ExplodingTNT eredita lo stato da TNT, ma acquisisce la capacità di essere annullata (Cancellable) e di essere eseguita come un thread (Runnable).

7.3 Esercizi e approfondimenti dal tutorato

Nel **Tutorato 3**, il concetto di interfaccia è centrale nell'esercizio della **Macchinetta delle bevande** e dei sistemi di pagamento. Questo esercizio mostra come disaccoppiare la logica della macchinetta dal tipo specifico di moneta o carta inserita, dipendendo esclusivamente da un'interfaccia.

7.3.1 Sistemi di pagamento polimorfici e Null Object Pattern

L'esercizio richiede di gestire diverse modalità di pagamento (Moneta, CartaDiCredito) che condividono la capacità di fornire un credito. Viene inoltre introdotto il **Pattern Null Object** tramite la classe NullCredito, che funge da "oggetto vuoto" sicuro per evitare fastidiose NullPointerException quando nessun pagamento è stato ancora inserito.


```

// 1. Interfaccia: definisce "cosa" un pagamento sa fare
public interface Credito {
    double getValore();
    boolean isValido();
}

// 2. Implementazione concreta: Moneta
public class Moneta implements Credito {
    private final double valore;

    public Moneta(double valore) {
        this.valore = valore;
    }

    @Override
    public double getValore() { return valore; }

    @Override
    public boolean isValido() { return valore > 0; }
}

// 3. Implementazione Null Object (Sostituisce il 'null')
public class NullCredito implements Credito {
    @Override
    public double getValore() { return 0.0; }

    @Override
    public boolean isValido() { return false; }
}

```

7.3.2 La Macchinetta (Contesto) e l'Esecuzione

La Macchinetta non sa nulla di monete o carte di credito. Conosce solo l'interfaccia Credito. Questo è il vero potere della programmazione basata sulle interfacce.

```
public class Macchinetta {
    // Inizializza in sicurezza usando il Null Object invece di 'null'
    private Credito creditoAttuale = new NullCredito();

    public void inserisciPagamento(Credito c) {
        if (c != null && c.isValido()) {
            this.creditoAttuale = c;
            System.out.println("Credito inserito: " + c.getValore() + "€");
        } else {
            System.out.println("Pagamento rifiutato o non valido.");
        }
    }

    public void erogaCaffe() {
        // Grazie al Null Object, non serve fare 'if (creditoAttuale != null)'
        if (creditoAttuale.getValore() >= 0.50) {
            System.out.println("Erogazione caffè in corso...");
            this.creditoAttuale = new NullCredito(); // Resetta il credito
        } else {
            System.out.println("Credito insufficiente!");
        }
    }
}

public class TestMacchinetta {
    public static void main(String[] args) {
        Macchinetta macchinetta = new Macchinetta();

        macchinetta.erogaCaffe(); // Tenta senza soldi: "Credito insufficiente"

        Credito dueEuro = new Moneta(2.00); // Polimorfismo di interfacce
        macchinetta.inserisciPagamento(dueEuro);
        macchinetta.erogaCaffe(); // Eroga e resetta
    }
}
```

Analisi didattica:

- **Disaccoppiamento:** Se un domani l'azienda decidesse di accettare pagamenti tramite Satispay, basterebbe creare class Satispay implements Credito. La classe Macchinetta non verrebbe toccata di una singola virgola (Principio Open/Closed).
- **Sicurezza (Null Object):** Inizializzare lo stato con new NullCredito() anziché null ci salva dai crash a runtime quando invochiamo i metodi per controllare il saldo, rendendo il codice della macchinetta molto più pulito e privo dei classici controlli != null.

8 Lezione 08: Composizione vs Ereditarietà e lo Strategy Pattern

8.1 Spiegazione teorica

Finora abbiamo utilizzato l'ereditarietà per condividere stato e comportamento, modellando la relazione "*is-a*" (è un). Tuttavia, l'ereditarietà in Java è singola (una classe può avere una sola superclasse) e intrinsecamente rigida.

- **I limiti dell'Ereditarietà:** Se volessimo modellare un **Piglin** che può attaccare con la spada, con la balestra o a mani nude, potremmo pensare di creare le sotto-classi **SwordPiglin**, **RangedPiglin** e **PunchPiglin**. Ma cosa succederebbe se a un **RangedPiglin** si rompesse la balestra a runtime? L'ereditarietà non ci permette di cambiare la classe di un oggetto durante l'esecuzione: un **RangedPiglin** rimarrebbe tale per sempre. Questo genera un'esplosione combinatoria di classi e una fortissima rigidità.
- **La Composizione ("has-a"):** La soluzione è favorire la *composizione* rispetto all'ereditarietà. Invece di dire "il Piglin è un tiratore", diciamo "il Piglin **ha un** metodo di attacco". L'oggetto principale viene composto da sotto-oggetti che ne definiscono il comportamento.
- **Lo Strategy Pattern:** È un design pattern comportamentale basato sulla composizione che permette di definire una famiglia di algoritmi (strategie), incapsularli in classi separate e renderli intercambiabili.
 - Si definisce un'Interfaccia comune per le strategie (es. **AttackStrategy**).
 - Si implementano le **Strategie Concrete** che rispettano il contratto (es. **PunchAttack**, **CrossbowAttack**).
 - La classe di **Contesto** (es. **Piglin**) mantiene un riferimento all'interfaccia e delega l'azione al sotto-oggetto, potendolo sostituire a runtime.

8.2 Implementazione del codice

Vediamo come il `Piglin` utilizzi lo Strategy Pattern per cambiare tipo di attacco dinamicamente senza mai dover cambiare la propria natura (classe).

8.2.1 L'Interfaccia Strategy (`AttackStrategy.java`)

```
package lecture08.strategy.strategies;

public interface AttackStrategy {
    // Contratto: tutte le strategie di attacco devono poter essere eseguite
    void execute();
}
```

Commento: Qualsiasi classe che implementa questa interfaccia dovrà fornire la propria logica per il metodo `execute()`.

8.2.2 Le Strategie Concrete

```
// Strategia 1: Attacco corpo a corpo
public class PunchAttack implements AttackStrategy {
    @Override
    public void execute() {
        System.out.println(">> Attack: *Punch* (Melee Hit)");
    }
}

// Strategia 2: Attacco a distanza
public class CrossbowAttack implements AttackStrategy {
    @Override
    public void execute() {
        System.out.println(">> Attack: *TWANG* (Fired Arrow)");
    }
}
```

Commento: La logica dell'attacco non è più cablata nel `Piglin`, ma è separata in classi altamente modulari. Se volessimo aggiungere un `AxeAttack`, basterebbe creare una nuova classe senza toccare nulla del codice esistente (Principio Open/Closed).

8.2.3 La Classe di Contesto (Piglin.java)

```
package lecture08.strategy;
import lecture08.strategy.strategies.*;

public class Piglin {
    // Relazione HAS-A: Il Piglin ha una strategia, non sa quale sia di preciso
    private AttackStrategy currentStrategy;

    public Piglin(){
        // Strategia di default alla creazione
        this.currentStrategy = new PunchAttack();
    }

    // Permette di cambiare la strategia A RUNTIME
    public void setStrategy(AccessStrategy newStrategy) {
        this.currentStrategy = newStrategy;
        System.out.println("Piglin changed strategy to "
                           + newStrategy.getClass().getSimpleName());
    }

    public void fight() {
        if (currentStrategy == null) {
            System.out.println("Piglin stands still (No Strategy).");
            return;
        }
        // DELEGAZIONE: Il Piglin non sa come fare danno, delega all'oggetto
        System.out.print("Piglin acts: ");
        currentStrategy.execute();
    }
}
```

Commento: Questo è il fulcro del pattern. Il metodo `fight()` non contiene `if/else` per capire che arma ha il Piglin, ma fa una semplice *delegazione* al sotto-oggetto tramite `currentStrategy.execute()`.

8.2.4 Modifica del comportamento a Runtime (Lecture8.main)

```
public static void strategyPatternExample() {
    Piglin monster = new Piglin();

    System.out.println("\n--- Round 1: Default Behavior ---");
    monster.fight(); // Eseguirà il PunchAttack

    System.out.println("\n--- Round 2: Picking up a crossbow ---");
    // Sostituiamo il sotto-oggetto: il Piglin cambia comportamento!
    monster.setStrategy(new CrossbowAttack());
    monster.fight(); // Eseguirà il CrossbowAttack
}
```

Commento: Se l'arma del Piglin si rompesse, basterebbe chiamare `monster.setStrategy(new PunchAttack())` e il comportamento ritornerebbe immediatamente quello corpo a corpo. L'ereditarietà non ci avrebbe mai permesso una simile flessibilità dinamica.

8.3 Esercizi e approfondimenti dal tutorato

Nel **Tutorato 3 e 4**, lo Strategy Pattern viene ampiamente utilizzato per gestire algoritmi intercambiabili legati alla logica di business, come ad esempio il calcolo di tariffe, sconti o danni, rimuovendo la necessità di fragili blocchi `if-else` nidificati.

8.3.1 Esercizio: Sistema di Sconti (Strategy Pattern)

Immaginiamo di dover calcolare il prezzo di un biglietto d'ingresso. Invece di cablare le regole di sconto (es. Studente, Anziano, Default) dentro la classe principale, incapsuliamo ogni formula matematica in una strategia dedicata.

```
// 1. Interfaccia della Strategia (Il "Cosa")
public interface StrategiaSconto {
    double applicaSconto(double prezzoOriginale);
}

// 2. Strategie Concrete (Il "Come")
public class ScontoStudente implements StrategiaSconto {
    @Override
    public double applicaSconto(double prezzo) {
        return prezzo * 0.80; // Applica il 20% di sconto
    }
}

public class NessunoSconto implements StrategiaSconto {
    @Override
    public double applicaSconto(double prezzo) {
        return prezzo; // Restituisce il prezzo pieno
    }
}
```

```
// 3. Classe di Contesto (Il Biglietto)
public class Biglietto {
    private final String evento;
    private final double prezzoBase;
    private StrategiaSconto strategiaSconto; // Relazione HAS-A

    public Biglietto(String evento, double prezzoBase) {
        this.evento = evento;
        this.prezzoBase = prezzoBase;
        this.strategiaSconto = new NessunoSconto(); // Strategia di default
    }

    // Permette di iniettare una nuova strategia a runtime
    public void setStrategiaSconto(StrategiaSconto nuovaStrategia) {
        this.strategiaSconto = nuovaStrategia;
    }

    public double getPrezzoFinale() {
        // Delega il calcolo matematico all'oggetto Strategia
        return strategiaSconto.appllicaSconto(this.prezzoBase);
    }
}
```

Analisi didattica:

- **Manutenibilità (Open/Closed Principle):** Se l'amministrazione decide di introdurre uno `ScontoBlackFriday`, il programmatore dovrà solo creare una nuova classe che implementa `StrategiaSconto`. Il file `Biglietto.java` non verrà sfiorato.
- **Dinamicità a Runtime:** Proprio come il Piglin cambiava arma durante il combattimento, qui l'utente può inserire un coupon nel carrello e il sistema aggiornerà il calcolo all'istante richiamando `biglietto.setStrategiaSconto(new ScontoCoupon())`.

9 Lezione 09 (Lab 1): Modellazione, Identità degli oggetti e gerarchie

9.1 Spiegazione teorica e Struttura del Laboratorio

Questa lezione rappresenta il primo di una serie di laboratori pratici. Nei laboratori, l'obiettivo si sposta dalla teoria alla *modellazione attiva* di un problema, per cominciare a prepararsi ai temi d'esame.

Il focus del Laboratorio 1 è il primo passo dell'Ingegneria del Software orientata agli oggetti. Data una specifica testuale, si deve procedere per fasi logiche:

- **Identificare le Classi e i Campi (Fase 1):** Quali entità diventano classi? Quali informazioni costituiscono lo "stato" di queste entità?
- **Modificatori di visibilità (Fase 2):** Quali campi devono essere `private` per garantire l'incapsulamento e gli invarianti? Quali metodi devono essere `public` per offrire un'interfaccia usabile?
- **Uso di `final` e `static` (Fase 3):** Quali informazioni non cambiano mai nel ciclo di vita dell'oggetto (`final`)? Quali appartengono alla classe in generale e non alla singola istanza (`static`)?
- **Identificazione delle Enum (Fase 4):** Quali concetti rappresentano un set finito di opzioni categoriche (anziché usare "magic numbers" o stringhe) e richiedono un'enumerazione?

Durante i laboratori, si discute in gruppo non tanto per scrivere subito l'algoritmo perfetto, ma per definire le *firme*, le gerarchie e lo *scheletro* architetturale.

9.2 Implementazione pratica: Esercizio di Modellazione

Simulando il lavoro di gruppo del Lab 1, proviamo a modellare l'identità di un'arma in un gioco:

```
// Fase 4: Identificazione delle Enum
public enum Rarity {
    COMMON, UNCOMMON, RARE, EPIC, LEGENDARY
}

public class Weapon {
    // Fase 3: Scelta dei modificatori final e static
    public static final int MAX_DURABILITY = 100; // Globale a tutte le armi

    // Fase 1 e 2: Definizione dei campi e della visibilità
    private final String name;           // Immutabile e nascosto
    private final Rarity rarity;         // Immutabile e nascosto
    private int durability;              // Mutevole, ma privato (Incapsulamento)

    public Weapon(String name, Rarity rarity) {
        this.name = name;
        this.rarity = rarity;
        this.durability = MAX_DURABILITY;
    }

    // API per manipolare lo stato in modo sicuro mantenendo gli invarianti
    public void use() {
        if (this.durability > 0) {
            this.durability--;
            System.out.println(this.name+"used. Durability:"+this.durability);
        } else {
            System.out.println(this.name + " is broken!");
        }
    }
}
```


Commento e Analisi: Questo snippet riassume l'obiettivo formativo del Lab 1. La classe `Weapon` nasconde il suo stato interno (`private`). Il nome e la rarità sono `final` perché l'identità dell'arma non cambia nel tempo. L'uso della `enum Rarity` scongiura l'uso di stringhe vulnerabili a errori di battitura. Infine, `MAX_DURABILITY` è stato reso `static` e `final` perché è una regola globale del gioco.

9.3 Esercizi e approfondimenti dal tutorato: Identità e ID Univoci

Nei primissimi tutorati e nel Laboratorio 1, un problema classico di modellazione riguarda la gestione dell'**Identità** di un oggetto. Quando creiamo molteplici istanze di una classe, come facciamo a distinguerle in modo univoco e sicuro (senza dipendere dall'indirizzo di memoria)?

La soluzione architetturale standard (molto richiesta in sede d'esame) prevede l'uso combinato di `static` e `final` per generare codici identificativi progressivi (es. matricole, numeri di conto o ID di database).

9.3.1 Esercizio: Generatore di Matricole (Studente)

L'obiettivo è creare una classe in cui ogni nuovo oggetto ottenga automaticamente una matricola univoca, senza che l'utente debba inserirla manualmente, e che tale matricola sia inalterabile.

```
public class Studente {
    // 1. Variabile STATICA: condivisa da tutti.
    // Funge da generatore globale (es. parte da 1000)
    private static int contatoreGlobale = 1000;

    // 2. Variabile FINAL e d'istanza: l'identità unica del singolo studente
    private final int matricola;
    private final String codiceFiscale;

    // 3. Variabile di stato mutabile (può cambiare corso)
    private String corsoDiLaurea;

    public Studente(String codiceFiscale, String corsoDiLaurea) {
        // Assegna il valore corrente e POI incrementa il contatore globale
        this.matricola = contatoreGlobale++;
        this.codiceFiscale = codiceFiscale;
        this.corsoDiLaurea = corsoDiLaurea;
    }

    public int getMatricola() {
        return this.matricola;
    }

    // NOTA BENE: Non esiste un setMatricola()!
    // L'identità, una volta assegnata nel costruttore, è blindata.
}
```

Analisi didattica per l'esame:

- **Gestione dello Stato Globale:** Il `contatoreGlobale` esiste e mantiene il suo valore a livello di Classe. Ogni volta che viene invocato il costruttore tramite `new`, la classe "stacca un nuovo biglietto" (incrementando il contatore), garantendo che non esistano mai due studenti con la stessa matricola nel programma.
- **Sicurezza dell'Identità:** Rendendo la `matricola` e il `codiceFiscale` dei campi `private final`, garantiamo a livello di compilatore che nessun pezzo di codice (nemmeno all'interno della classe stessa) possa alterarli dopo la fase di istanziamento. L'oggetto nasce e muore con quella precisa identità.

10 Lezione 10: Il Dispatch (Static vs Dynamic Binding) e la risoluzione degli argomenti

10.1 Spiegazione teorica

In Java, il processo che determina quale blocco di codice eseguire quando viene invocato un metodo si chiama **Dispatch** (o Binding). Per capirlo a fondo, dobbiamo distinguere due concetti fondamentali legati alle variabili:

- **Tipo Statico (Compile-time type):** È il tipo con cui la variabile è stata dichiarata nel codice (es. `Block b`). È l'unico tipo che il compilatore "vede". Determina *quali* metodi è possibile chiamare su quell'oggetto.
- **Tipo Dinamico (Runtime type):** È il tipo dell'oggetto reale che è stato allocato in memoria nella Heap tramite la keyword `new` (es. `= new DiamondBlock()`). Determina *quale implementazione* del metodo verrà effettivamente eseguita a runtime.

A partire da questa distinzione, Java applica due meccanismi di binding:

1. **Dynamic Binding (Late Binding / Dynamic Dispatch):** È il comportamento standard in Java. Quando si invoca un metodo su un oggetto, la Java Virtual Machine guarda il **tipo dinamico** dell'oggetto per decidere quale codice eseguire. Se la sottoclasse ha fatto l'overriding del metodo, verrà eseguita l'implementazione della sottoclasse.
2. **Static Binding (Early Binding / Static Dispatch):** Avviene a tempo di compilazione. Il compilatore collega la chiamata al metodo usando il **tipo statico** dell'oggetto. Questo accade obbligatoriamente (e unicamente) per i metodi dichiarati `final`, `static` e `private`, perché, per definizione, tali metodi non possono subire overriding.

Interazione tra Dispatch e Overloading (Risoluzione degli argomenti): L'overloading (stesso nome del metodo, parametri diversi) viene risolto **a tempo di compilazione**. Il compilatore decide quale variante (firma) del metodo chiamare basandosi esclusivamente sul **tipo statico** degli argomenti passati. Il *Dynamic Dispatch* si applica solo all'oggetto ricevente (quello alla sinistra del punto), mai ai suoi argomenti.

10.2 Implementazione del codice

10.2.1 Static vs Dynamic Binding (Block e DiamondBlock)

```
public class Block {
    // Metodo normale: soggetto a Dynamic Binding
    public void onBreak() {
        System.out.println("Generic Block breaks into pieces.");
    }

    // Metodo final: soggetto a Static Binding
    public final void getRegistryName() {
        System.out.println("ID: minecraft:generic_block");
    }
}

public class DiamondBlock extends Block {
    @Override
    public final void onBreak() {
        System.out.println("DiamondBlock: *CLING* (Drops Diamonds!)");
    }
    // L'override di getRegistryName() genererebbe un errore di compilazione
}
```

Test del Binding:

```
private static void predictBindingExample() {
    Block bd = new DiamondBlock();
    // Tipo statico di bd: Block
    // Tipo dinamico di bd: DiamondBlock

    bd.onBreak();
    // Dynamic Binding: si valuta il tipo dinamico (DiamondBlock).
    // Stampa: "DiamondBlock: *CLING* (Drops Diamonds!)"

    bd.getRegistryName();
    // Static Binding (a causa del final): si valuta il tipo statico (Block).
    // Stampa: "ID: minecraft:generic_block"
}
```

Commento: Benché `bd` contenga un `DiamondBlock`, il compilatore ci impedisce di invocare metodi esclusivi del diamante (come un ipotetico `getDiamond()`) perché il tipo statico `Block` non li possiede. L'IDE vi darebbe errore.

10.2.2 La risoluzione degli argomenti (Miner e Pick)

```
public class Miner {
    // Variante 1: ascia normale
    public void mine(Pick p) {
        System.out.println("-> Used Generic Mining (Slow)");
    }

    // Variante 2: ascia in diamante
    public void mine(DiamondPick p) {
        System.out.println("-> Used Diamond Mining (Instant!)");
    }
}

public static void argBindingExample() {
    Miner steve = new Miner();

    Pick hiddenDiamond = new DiamondPick();
    // Tipo statico dell'argomento: Pick
    // Tipo dinamico dell'argomento: DiamondPick

    steve.mine(hiddenDiamond);
}
```

Commento: L'invocazione `steve.mine(hiddenDiamond)` stamperà `-> Used Generic Mining (Slow)`. Perché non ha usato la variante superveloce? Perché il compilatore, nel momento in cui sceglie quale metodo in overloading usare, valuta il **tipo statico** dell'argomento (`Pick`). L'overloading si risolve in modo statico e ignora il reale contenuto dinamico della variabile.

10.3 Esercizi e approfondimenti dal tutorato: Prevedere il Binding

Negli esercizi visti a tutorato e tipici delle prove d'esame, viene spesso richiesto di prevedere l'output di un programma in cui *Overloading* e *Overriding* si mescolano. Per non farsi ingannare, bisogna applicare la rigorosa regola in due step.

10.3.1 Esercizio: Il tranello del Dispatch

Analizziamo il seguente codice e proviamo a prevederne l'output:

```
class A {
    void stampa(A obj) { System.out.println("A stampa A"); }
}

class B extends A {
    // OVERRIDING (stessa firma di A)
    @Override
    void stampa(A obj) { System.out.println("B stampa A"); }

    // OVERLOADING (firma diversa)
    void stampa(B obj) { System.out.println("B stampa B"); }
}

public class TestBinding {
    public static void main(String[] args) {
        A var1 = new B();
        B var2 = new B();

        var1.stampa(var2);
    }
}
```

Risoluzione passo-passo (La Regola d'Oro):

1. **Fase 1 (Compilazione e Staticità):** Si guarda esclusivamente il *tipo statico* della variabile che chiama il metodo (`var1` è di tipo `A`) e il *tipo statico* dell'argomento passato (`var2` è di tipo `B`). Il compilatore entra in `A` e cerca un metodo `stampa(B)`. Non lo trova! L'unica opzione compatibile in `A` (grazie al polimorfismo) è `stampa(A)`. Quindi, a livello di compilazione, la firma "congelata" e scelta in via definitiva è `stampa(A)`.
2. **Fase 2 (Esecuzione e Dinamicità):** A runtime, la JVM abbandona il tipo statico e guarda il *tipo dinamico* del chiamante (`var1` è fisicamente un oggetto `B` in memoria). Partendo da `B`, la JVM cerca l'implementazione più specifica per la firma congelata nella Fase 1 (ovvero `stampa(A)`). Poiché `B` ha fatto l'override di `stampa(A)`, verrà eseguita la versione di `B`.

Output finale: B stampa A

Attenzione: Moltissimi studenti all'esame sbagliano e rispondono `B stampa B`, perché guardano istintivamente solo i tipi dinamici ignorando che l'overloading si risolve sempre e solo staticamente nella Fase 1!

11 Lezione 11: V-Table, Casting, instanceof e Template Method

11.1 Spiegazione teorica

Questa lezione fa da ponte tra i concetti di basso livello (come la JVM gestisce la memoria per l'ereditarietà) e quelli di alto livello (Design Patterns per evitare codice fragile).

- **La V-Table (Virtual Table):** Abbiamo visto che il *Dynamic Dispatch* sceglie il metodo a runtime in base al tipo dinamico dell'oggetto. Ma come fa fisicamente? Ogni classe possiede una V-Table, ovvero una tabella (array) di puntatori alle locazioni di memoria dove risiedono le istruzioni dei suoi metodi. Quando un oggetto viene allocato nella *Heap*, la sua primissima parola di memoria è un puntatore nascosto alla V-Table della sua classe. Se invochiamo `miner.mine(p)`, Java va all'indirizzo dell'oggetto `miner`, segue il puntatore alla V-Table, salta all'offset corrispondente al metodo `mine` ed esegue il codice.
- **Casting e instanceof:** A volte il tipo statico di una variabile è generico (es. `Entity`), ma abbiamo l'esigenza di invocare un metodo specifico della sua classe reale (es. `hiss()` di `Creeper`).
 - **instanceof:** È un operatore che restituisce `true` se il tipo dinamico dell'oggetto a runtime è compatibile con la classe specificata.
 - **Casting (Downcasting):** Forza il compilatore a trattare una variabile generica come se fosse di un tipo più specifico (es. `(Creeper) myEntity`). Se ci sbagliamo e l'oggetto non è davvero di quel tipo, il programma va in crash a runtime con una `ClassCastException`.

Attenzione (Code Smell): Il professore sottolinea che l'uso massiccio di `instanceof` e casting è un sintomo di pessima progettazione a oggetti. Viola il Principio Open/Closed (se aggiungo una classe, devo scovare e modificare decine di `if/else` sparsi nel codice), è prone a errori ed è lento. La soluzione è sempre il **Polimorfismo**.

- **Il Pattern Template Method:** È un *Behavioral Pattern* che risolve proprio l'abuso di `instanceof`. Si applica quando diverse classi eseguono la stessa identica sequenza di operazioni (un algoritmo standard), ma differiscono nei dettagli di alcuni passaggi.
 - Si crea una classe astratta base.
 - Si definisce un metodo **scheletro** (il *Template Method*), contrassegnato come `final` affinché l'ordine delle operazioni non possa essere alterato.
 - Lo scheletro richiama altri metodi interni (`abstract` o `protected`) che le sottoclassi implementeranno per fornire il comportamento specifico del singolo step.

11.2 Implementazione del codice

11.2.1 Il problema: L'abuso di instanceof (BadBlock.java)

```
public class BadBlock {
    public void destroyBlock() {
        System.out.println("1. Spawning Particles...");
        System.out.print("2. Playing Sound: ");

        // CATTIVA PRATICA: Controlli manuali sul tipo (lento e fragile)
        if (this instanceof BadGlass) {
            System.out.println("*Tink* (Glass Sound)");
        } else {
            System.out.println("*Crack* (Default Stone Sound)");
        }

        System.out.print("3. Dropping: ");
        if (this instanceof BadDiamond) {
            BadDiamond diamond = (BadDiamond) this; // Casting
            diamond.dropSpecialDiamond();
        } else if (this instanceof BadGlass) {
            System.out.println("Nothing (Shattered)");
        } else {
            System.out.println("Cobblestone");
        }
    }
}
```

Commento: Questo codice è fragile. Se domani aggiungessimo un `BadWoodBlock`, dovremmo aprire questa classe e aggiungere altri `else if`. Le classi non sono indipendenti, violando le regole base dell'OOP.

11.2.2 La soluzione: Il Template Method (AbstractBlock.java)

```
public abstract class AbstractBlock {

    // IL TEMPLATE METHOD: È final, fissa l'algoritmo (lo scheletro)
    public final void destroyBlock() {
        spawnParticles();    // Step 1: Logica privata, fissa per tutti
        playBreakSound();    // Step 2: Sovrascrivibile (facoltativo)
        dropLoot();          // Step 3: Obbligatorio per le sottoclassi
    }

    private void spawnParticles() {
        System.out.println(" *Poof* (Particles spawned)");
    }

    // Comportamento di default che le sottoclassi possono cambiare (Hook)
    protected void playBreakSound() {
        System.out.println(" *Crack* (Sound played)");
    }

    // Comportamento astratto: ogni blocco DEVE dire cosa dropa
    protected abstract void dropLoot();
}
```

Commento: La sequenza delle operazioni (`destroyBlock`) è blindata dal `final`. Le sottoclassi non devono più preoccuparsi di *quando* fare un'azione, ma solo di *come* farla all'interno del proprio contesto.

11.2.3 Le implementazioni concrete

```
public class DiamondOre extends AbstractBlock {
    @Override
    protected void dropLoot() {
        System.out.println(" Dropped: 1x Diamond");
    }
}

public class GlassBlock extends AbstractBlock {
    @Override
    protected void dropLoot() {
        System.out.println(" Dropped: Nothing (It shattered)");
    }

    @Override
    protected void playBreakSound() {
        System.out.println(" *Tink* (Glass Sound)");
    }
}
```

Commento: Le classi figlie sono ora pulite ed estremamente focalizzate solo sulle proprie peculiarità.

11.3 Esercizi e approfondimenti a lezione

Per verificare la flessibilità del pattern, è stato richiesto di espanderlo per gestire i blocchi di legno. La potenza del *Template Method* sta nel permettere di creare livelli intermedi nella gerarchia senza intaccare lo scheletro originale.

```
// Livello intermedio: fissa il suono per TUTTI i legni, ma lascia
// astratto il drop
public abstract class AbstractWoodBlock extends AbstractBlock {
    @Override
    protected void playBreakSound() {
        System.out.println(" (Soft Wood Sound)");
    }
}

// Implementazione finale: l'albero specifico definisce solo il suo loot
public class OakBlock extends AbstractWoodBlock {
    @Override
    protected void dropLoot() {
        System.out.println(" -> Dropped: Oak Log");
    }
}
```

Conclusione: Se volessimo aggiungere un `CherryBlock`, basterebbe estendere `AbstractWoodBlock` e implementare una singola riga per il drop. Il codice madre del distruttore e della gestione del suono non verrebbe toccato, rispettando appieno il principio dell'ereditarietà.

11.4 Esercizi e approfondimenti dal tutorato: Il Downcasting Sicuro

Nei laboratori e nei tutorati intermedi (come il **Tutorato 4**), ci si imbatte spesso in situazioni in cui il tipo statico di una variabile è generico (es. `Dispositivo`), ma abbiamo la necessità assoluta di invocare un metodo esclusivo di una sua sottoclasse concreta (es. `connetti5G()` di `Smartphone`).

Riprendendo la teoria della Lezione 11, l'uso massiccio di controlli sul tipo è un *code smell*, ma qualora fosse strutturalmente indispensabile, il tutorato mostra come eseguire un **Downcasting sicuro** per evitare la temuta `ClassCastException`.

11.4.1 Esercizio: Verifica del Tipo Dinamico e Casting

Di seguito viene implementata una gerarchia di dispositivi e un metodo di smistamento all'interno di un centro di assistenza che valida il tipo reale dell'oggetto prima di forzarne il cambiamento di tipo.

```
// 1. Superclasse generica
public class Dispositivo {
    private final String marca;

    public Dispositivo(String marca) {
        this.marca = marca;
    }

    public String getMarca() { return marca; }
}

// 2. Sottoclasse con funzionalità esclusive
public class Smartphone extends Dispositivo {
    public Smartphone(String marca) {
        super(marca);
    }

    public void connetti5G() {
        System.out.println("Smartphone " + getMarca() + ": Connessione 5G attiva!");
    }
}
```

11.4.2 Il Controller di Assistenza e il test di robustezza

Nel gestore dei dispositivi, viene applicato l'operatore `instanceof` combinato con il casting esplicito.

```
public class CentroAssistenza {
    public static void attivaServiziAvanzati(Dispositivo d) {
        // 1. VERIFICA DI SICUREZZA: d è compatibile con Smartphone a runtime?
        if (d instanceof Smartphone) {
            // 2. DOWNCASTING: Forziamo in sicurezza il tipo statico verso il basso
            Smartphone sp = (Smartphone) d;

            // 3. Invocazione del metodo esclusivo della sottoclasse
            sp.connetti5G();
        } else {
            System.out.println("Dispositivo di marca " + d.getMarca() +
                               " non supporta il 5G.");
        }
    }
}
```

```

public static void main(String[] args) {
    Dispositivo generico = new Dispositivo("AnzianaMarca");
    Dispositivo mioTelefono = new Smartphone("GooglePixel");

    // Questo eseguirà il codice in sicurezza stampando il log del 5G
    attivaServiziAvanzati(mioTelefono);

    // Questo entrerà nell'else protetto, evitando il crash del programma
    attivaServiziAvanzati(generico);
}
}

```

Analisi didattica per l'esame:

- **Il rischio del cast cieco:** Se avessimo scritto direttamente `Smartphone sp = (Smartphone) generico;` senza il controllo condizionale dell'`instanceof`, il codice avrebbe compilato perfettamente (perché il compilatore non sa cosa conterrà la cella di memoria a runtime), ma l'applicazione sarebbe andata in crash immediato durante l'esecuzione sollevando una `ClassCastException`.
- **Evoluzione verso il Polimorfismo:** Come evidenziato dal docente a lezione, se in futuro dovessimo aggiungere nuove sottoclassi (es. `SmartTV`, `SmartWatch`), questo approccio ci costringerebbe a inserire una catena infinita di `if else instanceof`. La soluzione ingegneristica corretta consiste nell'usare il Polimorfismo o il pattern `Template Method` visto a lezione, spostando l'azione all'interno dei metodi d'istanza sovrascritti.

12 Lezione 12 (Lab 2): Classi astratte, Metodi e Refactoring

12.1 Spiegazione teorica e Struttura del Laboratorio

Questo secondo laboratorio sposta l'attenzione dalla semplice identificazione delle entità (vista nel Lab 1) alla **progettazione avanzata delle gerarchie e dei comportamenti**. L'obiettivo è abituarsi a fare scelte architetturali consapevoli in base al testo di un problema.

I gruppi di lavoro affrontano le seguenti fasi di modellazione:

- **Fase 1: Le classi astratte:** Non tutte le entità identificate in un testo devono poter essere istanziate direttamente. Capire quando una classe serve solo da "stampo base" e da collettore di comportamenti comuni (diventando quindi **abstract**) è essenziale per evitare l'istanziamento di oggetti logici incompleti o troppo generici.
- **Fase 2: I metodi, astratti e non:** All'interno di una classe astratta o di un genitore, il programmatore deve tracciare un confine netto: quali comportamenti hanno un'implementazione universale e condivisa da tutti i figli (metodi *concreti*), e quali invece rappresentano solo un contratto che *deve* essere specializzato da ogni figlio (metodi *astratti*).

- **Fase 3: Campi che dovrebbero essere metodi:** Un errore gravissimo e comunissimo nella progettazione è salvare come *stato* (in una variabile) un'informazione che è in realtà **derivabile** o calcolabile al volo da altri stati. Trasformare questi campi in metodi previene inconsistenze logiche.

12.2 Implementazione pratica: Esercizio di Refactoring

Per illustrare il cuore della "Fase 3", consideriamo un errore classico di modellazione e la sua risoluzione ingegneristica.

```
// MODELLAZIONE ERRATA: "isAlive" è un campo statico nello stato
public class BadPlayer {
    private int health = 20;
    private boolean isAlive = true; // Rischio altissimo di inconsistenza!

    public void takeDamage(int amount) {
        this.health -= amount;
        // Se non ci ricordiamo di aggiornare isAlive, il giocatore
        // avrà vita <= 0 ma risulterà ancora vivo per il resto del gioco.
        if (this.health <= 0) {
            this.isAlive = false;
        }
    }
}
```

```
// MODELLAZIONE CORRETTA: "isAlive" diventa un metodo (Proprietà calcolata)
public class GoodPlayer {
    private int health = 20;

    public void takeDamage(int amount) {
        this.health -= amount;
    }

    // Il campo sparisce e diventa un metodo:
    // valuta lo stato "in tempo reale", garantendo l'invariante logico.
    public boolean isAlive() {
        return this.health > 0;
    }
}
```

Commento e Analisi: Nel primo caso stiamo memorizzando informazioni ridondanti. Mantenere sincronizzati `health` e `isAlive` attraverso decine di metodi (danno, cure, pozioni, respawn) è prone a errori (*bug*). Nel secondo caso delegando l'informazione "vita" a un metodo derivato (`isAlive()`), si rende la classe a prova di errore e si riduce anche la memoria occupata (un `boolean` in meno per ogni oggetto allocato).

12.3 Esercizi e approfondimenti dal tutorato: Refactoring del Polimorfismo

Nei laboratori, uno degli errori di progettazione più penalizzati all'esame (spesso causa di bocciatura) è l'uso di costrutti `switch` o catene di `if/else` per determinare il comportamento di un oggetto in base al suo "tipo". Il laboratorio insegna a smantellare questo anti-pattern delegando la responsabilità alle sottoclassi.

12.3.1 Esercizio: Eliminazione degli `instanceof`

Vediamo la trasformazione di un gestore di suoni da un'architettura procedurale (sbagliata) a una puramente a oggetti (corretta).

```
// ANTI-PATTERN: Cattiva progettazione (Codice Fragile)
public class GestoreSuoniSbagliato {
    public void riproduciSuono(Blocco b) {
        // Accoppiamento forte e violazione dell'Open/Closed Principle
        if (b instanceof BloccoLegno) {
            System.out.println("Suono: *Toc Toc*");
        } else if (b instanceof BloccoVetro) {
            System.out.println("Suono: *Tink*");
        }
    }
}
```

Soluzione: Scomposizione polimorfica Estraiamo la logica creando una classe astratta genitore che definisce il contratto, e spostiamo l'implementazione del suono dentro le rispettive classi figlie.

```
// 1. La classe astratta definisce il contratto obbligatorio
public abstract class Blocco {
    public abstract void suona();
}

// 2. Le sottoclassi implementano la propria logica specifica
public class BloccoLegno extends Blocco {
    @Override
    public void suona() { System.out.println("Suono: *Toc Toc*"); }
}

public class BloccoVetro extends Blocco {
    @Override
    public void suona() { System.out.println("Suono: *Tink*"); }
}
```

```
// 3. Il Gestore diventa generico e invulnerabile alle modifiche
public class GestoreSuoniCorretto {
    public void riproduciSuono(Blocco b) {
        // Dynamic Dispatch: il compilatore non sa che blocco sia,
        // ma la JVM eseguirà il metodo corretto a runtime!
        b.suona();
    }
}
```

Analisi didattica per l'esame:

- **Rimozione del Code Smell:** Nel codice corretto non c'è più traccia di `instanceof` o casting. Il programma principale chiama semplicemente `b.suona()` disinteressandosi totalmente del tipo dinamico del blocco.
- **Scalabilità Estrema:** Se all'esame venisse chiesto di aggiungere `BloccoMetallo`, nel primo caso dovresti aprire e modificare la classe `GestoreSuoniSbagliato` (rischiando di rompere codice già testato). Nel secondo caso, ti basterebbe creare la nuova classe e il `GestoreSuoniCorretto` saprebbe già gestirla automaticamente.

13 Lezione 13: Gestione degli Errori ed Eccezioni

13.1 Spiegazione teorica

Nei linguaggi meno recenti (come il C), gli errori venivano gestiti tramite "codici di errore" (es. ritornare `-1` se un'operazione falliva). Questo approccio porta a numerosi problemi: come differenziare gli errori? Cosa succede se `-1` è un valore di ritorno lecito? Come separare la logica di business dalla gestione degli errori?

I linguaggi orientati agli oggetti risolvono questo problema elevandolo a livello semantico tramite le **Eccezioni**.

- **La gerarchia delle eccezioni:** In Java, tutte le eccezioni estendono la classe radice `Throwable`. Da essa derivano due grandi rami:
 - **Error:** Errori critici della JVM (es. `OutOfMemoryError`). Non vanno mai gestiti.
 - **Exception:** Errori recuperabili derivanti dalla logica dell'applicazione. Si dividono a loro volta in:
 - * **Checked Exceptions:** Il compilatore obbliga il programmatore a gestirle (tramite `try-catch` o dichiarandole nella firma del metodo).
 - * **Unchecked Exceptions (RuntimeException):** Eccezioni dovute a bug del programmatore (es. `NullPointerException`, `IndexOutOfBoundsException`). Queste **non** vanno gestite con un `try-catch`, ma prevenute sistemando il codice logico (es. controllando che l'indice non sia fuori dai limiti prima di accedere all'array).

- **Sintassi base (`throw` e `throws`):**

- `throw new ...`: Solleva materialmente l'eccezione, interrompendo l'esecuzione normale del metodo.
- `throws ...`: Si mette nella firma del metodo per dichiarare che il metodo "potrebbe" sollevare tali eccezioni, scaricando la responsabilità di gestirle a chi lo invoca.

- **Gestione (`try-catch`):** Se un metodo dichiara di lanciare un'eccezione checked, chi lo chiama deve racchiuderlo in un blocco `try`. Se l'eccezione si verifica, il flusso salta al blocco `catch` corrispondente. **Regola fondamentale:** i blocchi `catch` vanno ordinati dal più specifico (sottoclasse) al più generico (superclasse).

- **Eccezioni e Subtyping (Overriding):** Se una sottoclasse fa l'override di un metodo, può dichiarare (`throws`) solo le stesse eccezioni del metodo originale, un sottoinsieme di esse, oppure sottotipi di quelle originali. **Non può dichiarare eccezioni nuove o più generiche.** Questo garantisce che il polimorfismo non si rompa a runtime.

- **Try-with-resources:** Un costrutto speciale (es. `try (FileReader f = new FileReader(...)) { ... }`) introdotto per chiudere automaticamente le risorse (file, connessioni) al termine del blocco, anche in caso di errore. Funziona solo con oggetti che implementano l'interfaccia `AutoCloseable`.

13.2 Implementazione del codice

13.2.1 Definizione di una gerarchia di Eccezioni

Creare eccezioni personalizzate è utile per dare semantica precisa agli errori del nostro dominio (es. Minecraft).

```
// Eccezione base del nostro dominio
public class CraftingException extends Exception {
    public CraftingException(String msg) { super(msg); }
}

// Eccezioni specifiche figlie di CraftingException
public class RecipeMissingException extends CraftingException {
    public RecipeMissingException(String msg) { super(msg); }
}

public class InventoryFullException extends CraftingException {
    public InventoryFullException(String msg) { super(msg); }
}
```

13.2.2 Sollevare e dichiarare le eccezioni (CraftingTable.java)

```
public class CraftingTable {
    // La firma dichiara quali errori checked possono sfuggire
    // da questo metodo
    public Block craftAdvanced(String item) throws RecipeMissingException,
                                                InventoryFullException {

        if (item.equals("UnknownThing")) {
            // Generazione materiale dell'errore
            throw new RecipeMissingException("What is a " + item + "?");
        }

        boolean hasSpace = false;
        if (!hasSpace) {
            throw new InventoryFullException("No empty slots available.");
        }
        return new Block("Boat");
    }
}
```

13.2.3 Catturare e gestire gli errori

```
public static void exceptionsExample() {
    CraftingTable table = new CraftingTable();

    try {
        table.craftAdvanced("Diamond_Sword"); // Potrebbe fallire
    }
    catch (RecipeMissingException e) {
        System.out.println("Unknown Recipe: " + e.getMessage());
        System.out.println(">> Opening Recipe Book...");
    }
    catch (InventoryFullException e) {
        System.out.println("Inventory Full: " + e.getMessage());
        System.out.println(">> Please clear a slot.");
    }
    catch (Exception e) { // Catch generico di "salvataggio" sempre alla fine
        System.out.println("Critical Error: " + e.getMessage());
    }
}
```

Commento: L'uso del try-catch divide in modo netto la *business logic* (il tentativo di craftare nel try) dalla *error logic* (i vari catch).

13.2.4 Eccezioni e Costruttori

I costruttori non possono restituire valori, quindi le eccezioni sono l'unico modo per segnalare e bloccare la creazione di un oggetto in uno stato invalido.

```
public CraftingTable(int n) throws CraftingException {
    if (n < 0) {
        throw new CraftingException("Cannot create table with negative slots");
    }
}
```

13.3 Esercizi e approfondimenti dal tutorato (Lab 3)

Il **Laboratorio 3** e i tutorati associati si concentrano sulla trasformazione dei vincoli testuali in una gerarchia di eccezioni robuste. In sede d'esame, non è sufficiente "gestire" l'errore: bisogna "classificarlo" in modo che il *Controller* possa reagire in modo differenziato.

13.3.1 Esercizio: Progettazione di un sistema di Eccezioni

Se il testo d'esame specifica che "Un prelievo non può superare il saldo" o "Una missione richiede un livello minimo", la strategia corretta è creare una gerarchia che parte da un'eccezione astratta del dominio (es. *BankingException* o *QuestException*).

```
// 1. Eccezione base del dominio (Checked)
public abstract class QuestException extends Exception {
    public QuestException(String msg) { super(msg); }
}

// 2. Eccezioni specifiche: dettaglio del fallimento
public class InsufficientLevelException extends QuestException {
    public InsufficientLevelException(int livelloRichiesto) {
        super("Livello insufficiente! Richiesto: " + livelloRichiesto);
    }
}

public class AlreadyCompletedException extends QuestException {
    public AlreadyCompletedException() {
        super("Missione già conclusa in precedenza.");
    }
}
```

13.3.2 Gestione stratificata (Model-Controller)

Il codice deve separare la logica di controllo (lancio dell'eccezione nel *Model*) dalla logica di presentazione (cattura nel *Controller*).

```
// NEL MODEL (Logica pura)
public void startQuest(Quest q, int playerLevel) throws QuestException {
    if (q.isDone()) throw new AlreadyCompletedException();
    if (playerLevel < q.getMinLevel())
        throw new InsufficientLevelException(q.getMinLevel());
    // ... avvio missione
}

// NEL CONTROLLER (Logica di gestione)
public void handleStartQuest(Quest q) {
    try {
        model.startQuest(q, player.getLevel());
        view.showMessage("Missione avviata!");
    } catch (AlreadyCompletedException e) {
        view.showError("Ops! " + e.getMessage());
    } catch (InsufficientLevelException e) {
        view.showError("Requisiti non soddisfatti: " + e.getMessage());
    } catch (QuestException e) { // Catch generico (salvataggio)
        view.showError("Errore imprevisto: " + e.getMessage());
    }
}
```

Analisi didattica per l'esame:

- **Separazione delle responsabilità:** Il *Model* non deve mai stampare messaggi (niente `System.out.println` o `JOptionPane` nel *Model*). Lancia solo l'eccezione. È il *Controller* a decidere come notificare l'utente (tramite *view*).
- **Gerarchia dei Catch:** Si catturano prima le eccezioni più specifiche (es. `InsufficientLevelException`) e per ultima quella più generica (`QuestException`), garantendo che ogni errore riceva il messaggio dedicato.

14 Lezione 14: Modellazione e Spawner

14.1 Spiegazione teorica

In questa lezione il focus si sposta dalla singola classe alla progettazione di un sistema di gestione. Spesso nei software (specialmente nei videogiochi o nei sistemi di monitoraggio) non gestiamo oggetti isolati, ma intere popolazioni di entità che nascono, agiscono e muoiono.

- **Responsibility-Driven Design:** Bisogna decidere "chi fa cosa". Se abbiamo molte entità (come i mostri in Minecraft), non è efficiente che ognuna si gestisca da sola nel programma principale. Serve un oggetto "Manager", chiamato **Spawner**, che ha la responsabilità di:
 - Mantenere un elenco (collezione) di tutte le entità attive.
 - Creare nuove istanze e aggiungerle alla gestione.
 - Aggiornare lo stato di tutte le entità in un unico ciclo (*tick*).
- **Introduzione alle Collezioni (List):** Per gestire un numero variabile di entità, non usiamo gli array (che hanno dimensione fissa), ma le liste dinamiche di Java, come `ArrayList<T>`.
 - `List<Entity>`: È un primo esempio di **Generics**. Indica una lista che può contenere solo oggetti di tipo `Entity` o sottoclassi.
 - `entities.add(e)`: Aggiunge un elemento alla fine.
 - `for (Entity e : entities)`: Ciclo *for-each* per iterare su tutta la collezione.

14.2 Implementazione del codice

Vediamo il cuore dello **Spawner**, che utilizza il polimorfismo per aggiornare entità diverse senza conoscerne il tipo specifico.

14.2.1 La classe Spawner (Spawner.java)

```
package lecture14.spawn;
import java.util.ArrayList;
import java.util.List;

public class Spawner {
    // Collezione di entità gestite
    private final List<Entity> entities;

    public Spawner() {
        this.entities = new ArrayList<>();
    }

    // Aggiunge un'entità alla gestione
    public void spawn(Entity e) {
        this.entities.add(e);
    }

    // Aggiornamento globale (Tick)
    public void tick() {
        System.out.println("--- Spawner Tick ---");
        for (Entity e : entities) {
            // Polimorfismo: ogni entità agisce secondo la propria logica
            e.update();
        }
    }
}
```

Commento: Lo **Spawner** non sa se nella lista c'è un **Piglin** o uno **Zombie**. Sa solo che sono tutte **Entity** e che possiedono un metodo **update()**. Questa è la potenza del **Dynamic Binding** applicato alle collezioni.

14.3 Esercizi e approfondimenti dal tutorato

Il **Tutorato 4** approfondisce la modellazione di interi sistemi gerarchici tramite l'esercizio della **Sonda**. L'obiettivo è comprendere come un oggetto gestore (un **Satellite** o un centro di controllo, analogamente allo **Spawner** visto a lezione) possa supervisionare e coordinare una collezione dinamica di sensori polimorfici.

14.3.1 Modellazione di Sistemi Gerarchici (Sonda e SondaAvanzata)

Di seguito viene mostrata l'implementazione della struttura dati del dominio e dei comportamenti polimorfici sovrascritti nella sottoclasse.

```
// 1. Classe Base: Sonda
public class Sonda {
    private final String id;
    private double latitudine;
    private double longitudine;

    public Sonda(String id, double latitudine, double longitudine) {
        this.id = id;
        this.latitudine = latitudine;
        this.longitudine = longitudine;
    }

    public String getId() { return id; }

    // Metodo di business polimorfico
    public void eseguiRilevamento() {
        System.out.println("Sonda [" + id + "] alla posizione (" +
            latitudine + ", " + longitudine + "): Rilevamento base comp
    }
}

// 2. Sottoclasse Specializzata: SondaAvanzata (Aggiunge sensori termici)
public class SondaAvanzata extends Sonda {
    private double temperaturaRilevata;

    public SondaAvanzata(String id, double lat, double lon) {
        super(id, lat, lon);
        this.temperaturaRilevata = 0.0;
    }

    @Override
    public void eseguiRilevamento() {
        super.eseguiRilevamento(); // Esegue prima il comportamento base ereditato
        // Simula la lettura del sensore termico aggiuntivo
        this.temperaturaRilevata = 22.5;
        System.out.println(" -> [Sensore Termico] Temperatura rilevata: " +
            temperaturaRilevata + "°C");
    }
}
```

14.3.2 Il Gestore Centrale: Classe `Satellite`

La classe `Satellite` funge da manager dell'intera popolazione di sonde. Possiede una lista dinamica e aggiorna tutti i sotto-componenti in un unico ciclo di scansione globale.

```
import java.util.ArrayList;
import java.util.List;

public class Satellite {
    private final String nome;
    // Uso dei Generics per creare una collezione polimorfica sicura
    private final List<Sonda> sondeGestite;

    public Satellite(String nome) {
        this.nome = nome;
        this.sondeGestite = new ArrayList<>();
    }

    public void registraSonda(Sonda s) {
        this.sondeGestite.add(s);
    }

    // Aggiornamento globale (Analogo al metodo tick dello Spawner)
    public void avviaScansioneGlobale() {
        System.out.println("=== Satellite " + nome + ": Inizio scansione di rete ===");
        for (Sonda s : sondeGestite) {
            // Sfrutta il Dynamic Binding: ogni sonda risponde in base al suo tipo di
            s.eseguiRilevamento();
        }
        System.out.println("=====");
    }

    public static void main(String[] args) {
        Satellite controlCenter = new Satellite("Alpha-1");

        // Registrazione polimorfica di sonde di tipo diverso
        controlCenter.registraSonda(new Sonda("S-01", 46.06, 11.12));
        controlCenter.registraSonda(new SondaAvanzata("SA-99", 45.46, 9.19));

        // Avvio del monitoraggio sincronizzato
        controlCenter.avviaScansioneGlobale();
    }
}
```

Analisi architetturale per l'esame:

- **Responsibility-Driven Design:** L'esercizio evidenzia l'importanza di non spartigliare il controllo del flusso nel `main`. Il programma principale non deve scorrere singolarmente le sonde; tale responsabilità è delegata interamente all'oggetto manager `Satellite`, centralizzando e ripulendo l'architettura.
- **Estensibilità e Manutenibilità:** Grazie al `Dynamic Binding` applicato alle collezioni parametriche (`List<Sonda>`), l'aggiunta futura di nuove tipologie di sensori (es. `SondaBarometrica`) non richiederà alcuna modifica alla classe `Satellite`, preservando la stabilità del codice ed evitando costrutti condizionali fragili.

15 Lezione 15: Creazione ed Evoluzione (Factory Method e State Pattern)

15.1 Spiegazione teorica

Fino a questo momento abbiamo creato gli oggetti usando direttamente la keyword `new` (es. `new Zombie()`). Tuttavia, l'uso diretto di `new` accoppia fortemente (lega) il nostro codice alla classe concreta. Cosa succede se vogliamo che uno `Spawner` generi mostri diversi in base al livello di difficoltà, senza dover riempire il codice di `if/else`? E cosa succede quando un'entità cambia drasticamente comportamento nel tempo (es. un Creeper che si innesca)?

Per risolvere queste sfide architetturali, la *Gang of Four (GoF)* ha definito due pattern cruciali:

- **Factory Method Pattern (Creazionale):** Risolve il problema della creazione degli oggetti. Invece di chiamare `new` direttamente, si delega l'istanziatura a un metodo speciale (il *Factory Method*).
 - Permette a una superclasse di definire un'interfaccia per la creazione di un oggetto, ma lascia alle sottoclassi la decisione su *quale* classe concreta istanziare.
 - Favorisce il polimorfismo fin dal momento della nascita dell'oggetto.
- **State Pattern (Comportamentale):** Risolve il problema di un oggetto il cui comportamento cambia in base al suo stato interno.
 - Invece di usare variabili booleane (`isAngry`, `isSleeping`) e riempire i metodi di `if(isAngry) { ... }`, lo stato stesso diventa un **Oggetto** (simile allo *Strategy Pattern*).
 - L'oggetto principale (Contesto) delega l'esecuzione delle azioni all'oggetto "Stato" corrente. Quando lo stato logico cambia, il Contesto sostituisce il proprio oggetto Stato con uno nuovo.

15.2 Implementazione del codice

15.2.1 Il Factory Method (MobSpawner)

Immaginiamo di avere un gioco con diverse zone (Normale e Nether). Vogliamo uno spawner generico, ma che generi mostri appropriati per la zona.

```
// La classe creatrice astratta
public abstract class MobSpawner {
    // IL FACTORY METHOD: delega la creazione alle sottoclassi
    protected abstract Entity createMob();

    // Metodo logico comune che usa il Factory Method
    public void spawnWave() {
        Entity mob = createMob(); // Non so quale mob sia,
                                   // ma so che è un'Entity
        mob.spawnInWorld();
        System.out.println("Spawned a new"+mob.getClass().getSimpleName());
    }
}

// Fabbrica concreta 1
public class OverworldSpawner extends MobSpawner {
    @Override
    protected Entity createMob() {
        return new Zombie(); // Sceglie di creare uno Zombie
    }
}

// Fabbrica concreta 2
public class NetherSpawner extends MobSpawner {
    @Override
    protected Entity createMob() {
        return new Piglin(); // Sceglie di creare un Piglin
    }
}
```

Commento: Se aggiungiamo una nuova dimensione (es. The End), non dobbiamo modificare il codice esistente. Creiamo semplicemente un **EndSpawner** che restituisce un **Enderman**. Principio Open/Closed rispettato perfettamente.

15.2.2 Lo State Pattern (Creeper)

Un Creeper ha due stati: "Idle" (Calmo) e "Ignited" (Innescato). A seconda dello stato, reagisce diversamente al tempo che passa (tick).

```
// 1. L'Interfaccia dello Stato
public interface CreeperState {
    void tick(Creeper context);
}

// 2. Stato Concreto: Calmo
public class IdleState implements CreeperState {
    @Override
    public void tick(Creeper context) {
        System.out.println("Creeper is wandering around aimlessly...");
        // Se il giocatore è vicino, cambia stato!
        if (context.isPlayerNear()) {
            System.out.println("*Hiss* Creeper is ignited!");
            context.setState(new IgnitedState()); // Transizione di stato
        }
    }
}

// 3. Stato Concreto: Innescato
public class IgnitedState implements CreeperState {
    private int fuse = 3;

    @Override
    public void tick(Creeper context) {
        fuse--;
        if (fuse <= 0) {
            System.out.println("BOOM! Creeper exploded!");
            context.die();
        } else {
            System.out.println("Creeper is swelling up... " + fuse);
        }
    }
}
```

```
// 4. Il Contesto (Il Creeper stesso)
public class Creeper extends Entity {
    private CreeperState currentState;

    public Creeper() {
        this.currentState = new IdleState(); // Stato iniziale
    }

    public void setState(CreeperState newState) {
        this.currentState = newState;
    }

    @Override
    public void update() {
        // Delega il comportamento allo stato corrente
        currentState.tick(this);
    }
}
```

Commento: Senza lo State Pattern, il metodo `update()` del Creeper sarebbe un enorme e confuso blocco `if-else`. Con questo pattern, ogni classe di stato si concentra solo sulle proprie regole, rendendo l'evoluzione del comportamento pulita e modulare.

15.3 Esercizi e approfondimenti dal tutorato: Gestione di Stati Transizionali

Nel **Tutorato 5**, lo State Pattern viene approfondito per modellare sistemi del mondo reale regolati da transizioni vincolate, come il ciclo di funzionamento di una **Macchinetta del Caffè**. Rispetto all'esempio del **Creeper** visto a lezione, questo esercizio mostra come gestire input esterni ed espliciti dell'utente che scatenano cambi di stato protetti dalle regole del dominio.

15.3.1 Esercizio: La Macchinetta del Caffè (State Pattern)

Invece di riempire il contesto di variabili booleane (es. `hasMoney`, `isReady`) e controlli condizionali nidificati, deleghiamo interamente il comportamento a oggetti stato separati.

```
// 1. Interfaccia dello Stato: definisce le azioni permesse sul contesto
public interface StatoMacchinetta {
    void inserisciMoneta(MacchinettaCaffe macchina);
    void erogaCaffe(MacchinettaCaffe macchina);
}

// 2. Stato Concreto: Pronta (In attesa di moneta)
public class StatoPronta implements StatoMacchinetta {
    @Override
    public void inserisciMoneta(MacchinettaCaffe macchina) {
        System.out.println("Moneta accettata. Ora puoi richiedere il caffè.");
        macchina.setStatoCorrente(new StatoMonetaInserita()); // Transizione
    }

    @Override
    public void erogaCaffe(MacchinettaCaffe macchina) {
        System.out.println("Errore: Inserisci prima una moneta!");
    }
}

// 3. Stato Concreto: Moneta Inserita
public class StatoMonetaInserita implements StatoMacchinetta {
    @Override
    public void inserisciMoneta(MacchinettaCaffe macchina) {
        System.out.println("Moneta già presente! Rifiutata e restituita.");
    }

    @Override
    public void erogaCaffe(MacchinettaCaffe macchina) {
        System.out.println("Caffè in erogazione... Successo!");
        macchina.setStatoCorrente(new StatoPronta()); // Ritorna allo stato iniziale
    }
}
```

15.3.2 Il Contesto della Macchinetta

La classe di contesto mantiene il riferimento allo stato corrente e vi delega le operazioni di business passando se stessa (`this`) come parametro di contesto.

```
public class MacchinettaCaffe {
    private StatoMacchinetta statoCorrente;

    public MacchinettaCaffe() {
        this.statoCorrente = new StatoPronta(); // Stato iniziale di default
    }

    public void setStatoCorrente(StatoMacchinetta nuovoStato) {
        this.statoCorrente = nuovoStato;
    }

    // Metodi di interfaccia pubblica (Delegazione pura)
    public void inserisciMoneta() {
        statoCorrente.inserisciMoneta(this);
    }

    public void richiediCaffe() {
        statoCorrente.erogaCaffe(this);
    }

    public static void main(String[] args) {
        MacchinettaCaffe macchina = new MacchinettaCaffe();

        macchina.richiediCaffe();    // Output: Errore: Inserisci prima una moneta!
        macchina.inserisciMoneta();  // Output: Moneta accettata.
        macchina.richiediCaffe();    // Output: Caffè in erogazione... Successo!
    }
}
```

Analisi didattica per l'esame:

- **Prevenzione degli Stati Invalidi:** Senza questo pattern, per impedire l'erogazione fraudolenta o senza moneta avremmo dovuto inserire controlli `if (!hasMoney)` in ogni metodo. Con lo State Pattern, l'azione illegale è strutturalmente impossibile da eseguire perché la classe `StatoPronta` contiene intrinsecamente la logica d'errore per quel determinato momento del ciclo di vita dell'oggetto.
- **Principio Open/Closed:** Ogni classe si occupa rigorosamente delle proprie transizioni. Se all'esame venisse richiesto di estendere il sistema aggiungendo uno `StatoMacchinaGuasta`, basterà creare una nuova classe che implementa `StatoMacchinetta` e aggiornare le transizioni necessarie, senza dover toccare o rischiare di compromettere il funzionamento degli stati preesistenti.

16 Lezione 16 (Lab 3): Esercitazione su eccezioni, generics e pattern

16.1 Spiegazione teorica e Struttura del Laboratorio

Questo terzo laboratorio rappresenta un punto di sintesi fondamentale: l'obiettivo è imparare a unire i concetti architetturali (ereditarietà e interfacce) con la gestione della sicurezza (eccezioni) e la generalizzazione del codice (generics).

Quando si progetta un sistema complesso e robusto (come richiesto in sede d'esame), il programmatore deve padroneggiare:

- **Robustezza (*Fail-fast*):** Ogni operazione che altera lo stato del sistema deve essere rigidamente protetta. Se un'operazione non è valida, il metodo non deve limitarsi a fallire in silenzio (restituendo `false` o `null`), ma deve **lanciare un'eccezione** chiara ed esplicativa.
- **Generalizzazione (Generics):** Se una classe deve gestire collezioni o operazioni su più tipi di oggetti, va resa parametrica (usando `<T>`). Questo permette di riutilizzare la stessa logica di gestione (es. un `Manager<T>`) per entità diverse, garantendo al contempo che il compilatore controlli i tipi corretti (*Type Safety*).
- **Gestione del flusso d'errore:** Imparare dove istanziare (`throw`) l'eccezione (solitamente nel *Model*, ovvero nelle classi base) e dove catturarla (`catch`) (solitamente nel *Controller* o nella *View*) per informare l'utente finale.

16.2 Implementazione pratica: Classe Generica Sicura

Di seguito un classico schema di laboratorio che unisce Generics ed Eccezioni per creare un contenitore con capienza massima, utile per inventari, party di giocatori o registri.

```
// 1. Definizione dell'eccezione di dominio (Checked)
public class CapacityExceededException extends Exception {
    public CapacityExceededException(String message) {
        super(message);
    }
}

// 2. Classe Parametrica (Generics)
public class SafeManager<T> {
    private final List<T> items;
    private final int maxCapacity;

    public SafeManager(int maxCapacity) {
        this.items = new ArrayList<>();
        this.maxCapacity = maxCapacity;
    }

    // Il metodo dichiara esplicitamente la possibilità di fallire
    public void add(T item) throws CapacityExceededException {
        if (items.size() >= maxCapacity) {
            // Regola di business violata: lancia eccezione Checked
            throw new CapacityExceededException("Capienza massima raggiunta!");
        }
        if (item == null) {
            // Errore del programmatore: lancia eccezione Unchecked
            throw new IllegalArgumentException("L'elemento non può essere null.");
        }
        items.add(item);
    }
}
```

16.3 Esercizi e approfondimenti dal tutorato: Il Sistema QuestLog Parametrico

Nel **Tutorato 6**, l'applicazione combinata di *Generics* ed *Eccezioni di Dominio* si concretizza nel complesso sistema gestionale delle **Missioni (Quests)** per il personaggio **Geraldo**.

L'obiettivo dell'esercizio è creare un registro delle missioni (**QuestLog**) che sia generico e parametrico (per poter ospitare diverse tipologie di quest, come **MainQuest** o **DLCQuest**) ma rigidamente protetto dal lancio di eccezioni checked specifiche anziché da fragili controlli condizionali che restituiscono **false**.

16.3.1 Esercizio: Struttura delle Eccezioni e dell'Interfaccia Quest

Definiamo prima la gerarchia delle eccezioni e il contratto formale delle missioni che dovranno validare il livello di **Geraldo**.

```

// 1. Eccezioni di dominio specifiche per le Quest
public class QuestException extends Exception {
    public QuestException(String msg) { super(msg); }
}

public class AlreadyCompletedException extends QuestException {
    public AlreadyCompletedException() {
        super("Missione già contrassegnata come conclusa!");
    }
}

public class InsufficientLevelException extends QuestException {
    public InsufficientLevelException(int req) {
        super("Livello insufficiente! Richiesto livello: " + req);
    }
}

// 2. Interfaccia formale del Modello
public interface Quest {
    String getTitolo();
    int getLivelloRichiesto();
    void completa(int livelloGiocatore) throws QuestException;
}

```

16.3.2 Il Registro delle Missioni con i Generics (QuestLog<T>)

Implementiamo la classe logica QuestLog. Utilizziamo i Generics con un vincolo di tipo (*Bounded Type Parameter* <T extends Quest>) per garantire che il registro possa contenere solo oggetti che rispettano il contratto dell'interfaccia Quest.

```
import java.util.ArrayList;
import java.util.List;

public class QuestLog<T extends Quest> {
    private final List<T> activeQuests;

    public QuestLog() {
        this.activeQuests = new ArrayList<>();
    }

    public void addQuest(T quest) {
        // Il metodo contains baserà il controllo sul metodo equals() di Quest
        if (!activeQuests.contains(quest)) {
            activeQuests.add(quest);
        }
    }

    // Il metodo propaga l'eccezione verso il Controller superiore
    public void completaQuestNelLog(String titolo, int livelloGeraldo)
        throws QuestException {
        for (T q : activeQuests) {
            if (q.getTitolo().equals(titolo)) {
                // Delegazione polimorfica: controlla i vincoli e muta lo stato
                q.completa(livelloGeraldo);
                return;
            }
        }
        throw new QuestException("Quest '" + titolo + "' non trovata nel log.");
    }
}
```

Analisi del design e validità per l'esame:

- **Fail-Fast e Tracciabilità:** Invece di usare infiniti if che restituiscono false nel metodo di completamento (lasciando il programma all'oscuro del *perché* l'operazione sia fallita), il lancio di eccezioni specifiche permette al QuestController superiore di intercettare il problema esatto in un blocco try-catch e mostrare alla vista l'errore semantico corretto (es. "Livello insufficiente!").
- **Type Safety tramite i Generics:** L'uso del parametro <T extends Quest> assicura che il compilatore blocchi sul nascere qualsiasi tentativo di inserire nel log oggetti estranei al sistema di gioco. Al contempo, permette di riutilizzare la stessa identica struttura dati del QuestLog sia per le missioni principali della storia che per i contratti secondari, massimizzando il riutilizzo del codice senza ricorrere a casting pericolosi.

17 Lezione 17: Uguaglianza e Ordinamento

17.1 Spiegazione teorica

In Java, confrontare due oggetti o metterli in ordine richiede molta più attenzione rispetto all'utilizzo dei classici operatori matematici sui tipi primitivi. Dobbiamo distinguere due concetti di uguaglianza e comprendere i contratti previsti dalle interfacce di sistema.

- **Identità (Operatore ==):** L'operatore `==` verifica se due riferimenti puntano **esattamente allo stesso indirizzo di memoria** (cioè se sono lo stesso identico oggetto nello *Heap*).
- **Uguaglianza Logica (Metodo `equals()`):** Verifica se due oggetti, pur vivendo in due locazioni di memoria differenti, hanno "lo stesso valore" secondo la logica del nostro dominio (es. due blocchi di terra con le stesse coordinate, o due giocatori con lo stesso username). Di default, la classe base `Object` implementa `equals()` usando semplicemente `==`. Per ottenere l'uguaglianza logica, dobbiamo fare l'**override** del metodo.
- **Il contratto di `hashCode()`:** Se facciamo l'override di `equals()`, **dobbiamo obbligatoriamente** fare l'override anche di `hashCode()`. Questo metodo restituisce un numero intero usato per indicizzare l'oggetto in strutture dati ad alte prestazioni come `HashSet` o `HashMap`. La regola fondamentale di Java impone che: *"Se due oggetti sono logicamente uguali tramite `equals()`, allora i loro `hashCode()` devono restituire lo stesso numero"*. Se rompiamo questo patto, l'oggetto non verrà mai trovato all'interno delle collezioni.
- **Interfacce per l'Ordinamento:**
 - **`Comparable<T>`:** Si usa per definire l'**ordinamento naturale** della classe (es. ordine alfabetico per le Stringhe). La classe stessa implementa questa interfaccia e fornisce il metodo `compareTo(T o)`. Restituisce un numero negativo (se `this < o`), zero (se uguali), o positivo (se `this > o`).
 - **`Comparator<T>`:** È un oggetto esterno (spesso una classe anonima o lambda) che permette di definire **ordinamenti alternativi** al volo (es. ordinare i giocatori prima per punteggio, e a parità di punteggio per nome alfabeticamente), agendo in modo identico allo *Strategy Pattern*.

17.2 Implementazione del codice

17.2.1 Override sicuro di equals e hashCode

Vediamo come si implementa la logica per considerare uguali due oggetti Point2D.

```
import java.util.Objects;

public class Point2D {
    private final int x, y;
    public Point2D(int x, int y) {
        this.x = x; this.y = y;
    }
    @Override
    public boolean equals(Object obj) {
        // 1. Controllo di identità (ottimizzazione)
        if (this == obj) return true;
        // 2. Controllo di validità del tipo e null (evita ClassCastException)
        if (obj == null || this.getClass() != obj.getClass()) return false;

        // 3. Casting sicuro al tipo dinamico corretto
        Point2D other = (Point2D) obj;

        // 4. Confronto logico dello stato
        return this.x == other.x && this.y == other.y;
    }
    @Override
    public int hashCode() {
        // Usiamo il metodo utility di Java per generare un hash coerente
        return Objects.hash(x, y);
    }
}
```

17.2.2 Implementazione di Comparable (Ordinamento Naturale)

```
package lecture17.sorting;

public class LootItem implements Comparable<LootItem> {
    String name;
    private int goldValue;

    public LootItem(String name, int value) {
        this.name = name;
        this.goldValue = value;
    }

    public String getName() { return name; }
    public int getValue() { return goldValue; }

    @Override
    public int compareTo(LootItem other) {
        // Ordinamento naturale: decrescente per valore in oro
        return other.goldValue - this.goldValue;
    }

    @Override
    public String toString() {
        return name + " (" + goldValue + "g)";
    }
}
```

17.2.3 Utilizzo di Comparator (Ordinamento Alternativo a Classi Esterne)

Come da indicazione del corso, quando la logica di ordinamento differisce da quella naturale, si deve evitare l'uso di classi anonime *inline* e si deve creare una **classe esterna dedicata**. Questo favorisce la riusabilità, la separazione delle responsabilità ed è il design richiesto in sede d'esame.

1. La classe dell'ordinatore esterno (NameSorter.java):

```
package lecture17.sorting;

import java.util.Comparator;

public class NameSorter implements Comparator<LootItem> {
    @Override
    public int compare(LootItem o1, LootItem o2) {
        // Sfrutta il compareTo delle Stringhe per l'ordine alfabetico
        return o1.name.compareTo(o2.name);
    }
}
```

2. L'utilizzo pratico nell'applicazione:

```
import java.util.ArrayList;
import java.util.List;
import java.util.Comparator;

public class InventoryManager {
    public void sortAlphabetically() {
        List<LootItem> inventory = new ArrayList<>();
        inventory.add(new LootItem("Stick", 1));
        inventory.add(new LootItem("Diamond", 100));
        inventory.add(new LootItem("Coal", 1));

        // LootItem mantiene il suo ordinamento naturale (per valore)
        // Ma NameSorter permette di ordinarli alfabeticamente "al volo"
        Comparator<LootItem> nameSorter = new NameSorter();

        // Ordiniamo la lista usando l'ordinatore esterno dedicato
        inventory.sort(nameSorter);

        for (LootItem item : inventory) {
            System.out.println(item);
        }
    }
}
```

Commento: In questo blocco stiamo iniettando una logica esterna in `inventory.sort()`, scavalcando il `compareTo` di default della classe `LootItem`. Estrarre la logica in una classe separata (`NameSorter`) rende il codice più pulito, modulare e testabile. Nelle lezioni successive vedremo come questa sintassi si evolverà ulteriormente (e si semplificherà) con l'introduzione delle Lambda Expressions.

17.3 Esercizi e approfondimenti dal tutorato: Uguaglianza e Ordinamento pratico

Nei laboratori finali (come il **Tutorato 6** sul sistema `QuestController`), la corretta stesura del metodo `equals()` è uno degli scogli principali in sede d'esame. Quando il testo richiede: *"Il log non deve accettare missioni duplicate"*, il controllo si esegue invocando `activeQuests.contains(quest)`.

Se si omette l'override, `contains()` userà l'uguaglianza di identità ereditata da `Object` (`==`), finendo per inserire missioni identiche (ma allocate in aree di memoria diverse) e violando i requisiti del tema d'esame.

17.3.1 Esercizio: Uguaglianza Logica nelle Quest

Implementiamo `equals` e `hashCode` nella classe base delle missioni, stabilendo la regola di dominio secondo cui due missioni sono logicamente uguali se hanno lo stesso identico titolo.

```
package model.quests;
import java.util.Objects;

public abstract class AbstractQuest implements Quest {
    private final String title;
    private final int minLevel;

    public AbstractQuest(String title, int minLevel) {
        this.title = title;
        this.minLevel = minLevel;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true; // Ottimizzazione: stesso indirizzo
        // Se l'oggetto è nullo o non è compatibile con l'interfaccia Quest
        if (obj == null || !(obj instanceof Quest)) return false;

        Quest other = (Quest) obj;
        // L'identità logica della missione è dettata esclusivamente dal titolo
        return this.title.equals(other.getTitolo());
    }

    @Override
    public int hashCode() {
        return Objects.hash(title); // Genera hash coerente con equals
    }

    public String getTitolo() { return this.title; }
    public int getLivelloRichiesto() { return this.minLevel; }
}
```

17.3.2 Ordinamento del QuestLog

All'esame viene spesso richiesto di permettere all'utente di stampare o visualizzare gli elementi ordinati secondo criteri secondari (es. per livello di difficoltà crescente). Creiamo un `Comparator` esterno dedicato.

```
import java.util.Comparator;

// Ordinatore esterno: disaccoppia la logica di sorting dal Modello base
public class QuestLevelSorter implements Comparator<Quest> {
    @Override
    public int compare(Quest q1, Quest q2) {
        // Usa il comparatore nativo degli Integer per un ordinamento crescente
        return Integer.compare(q1.getLivelloRichiesto(), q2.getLivelloRichiesto());
    }
}
```

Analisi didattica per l'esame:

- **Sicurezza delle Collezioni:** Avendo fatto l'override accoppiato di `equals()` e `hashCode()`, ora possiamo inserire le missioni in strutture dati ad alte prestazioni (come un `HashSet<Quest>`) o usare `List.contains()` con la totale certezza matematica che i duplicati logici verranno respinti.
- **Flessibilità Architetture:** Invocando `activeQuests.sort(new QuestLevelSorter())` all'interno del *Controller*, possiamo riordinare in tempo reale l'interfaccia grafica per l'utente, senza "sporcare" la classe `Quest` con logiche di ordinamento che potrebbero cambiare frequentemente.

18 Lezione 18: Comportamenti reattivi e operazioni esterne (Observer e Visitor)

18.1 Spiegazione teorica

Man mano che i sistemi diventano complessi, si presentano due problemi ricorrenti: come far comunicare oggetti diversi senza accoppiarli (senza che debbano conoscersi intimamente) e come aggiungere nuove operazioni a una gerarchia di classi senza doverla modificare continuamente. La soluzione risiede in due Design Pattern comportamentali avanzati.

- **Observer Pattern (Comportamenti Reattivi):** Risolve il problema della notifica di stato. Definisce una dipendenza "uno-a-molti" tra oggetti in modo che, quando un oggetto (*Subject* o *Observable*) cambia il suo stato interno, tutti gli oggetti dipendenti (*Observers*) vengano notificati e aggiornati automaticamente.
 - Consente un **accoppiamento debole** (loose coupling): il *Subject* sa solo che i suoi osservatori implementano una certa interfaccia, non conosce le loro classi concrete.
 - È il cuore dei sistemi ad eventi (es. listener dei bottoni nelle interfacce grafiche) e dell'architettura MVC che vedremo in seguito.
- **Visitor Pattern (Operazioni Esterne):** Risolve il problema di dover aggiungere nuove operazioni su una struttura di oggetti eterogenei senza alterare le classi degli oggetti stessi.
 - Utilizza una tecnica chiamata **Double Dispatch**: l'operazione da eseguire dipende sia dal tipo del *Visitor* (l'operazione) sia dal tipo dell'*Element* (l'oggetto che riceve la visita).
 - Si usa quando la gerarchia degli elementi è molto stabile (es. la gerarchia dei blocchi o delle entità), ma le operazioni da eseguire su di essi cambiano o si espandono frequentemente (es. calcolare danni speciali, esportare dati in file, renderizzare graficamente).

18.2 Implementazione del codice

18.2.1 L'Observer Pattern (Player e AchievementSystem)

Vogliamo sbloccare un achievement quando il giocatore raccoglie i diamanti, ma non vogliamo che la classe `Player` contenga direttamente il codice degli achievement (violazione della singola responsabilità).

```
import java.util.ArrayList;
import java.util.List;

// 1. Interfaccia Observer (Chi ascolta)
public interface PlayerObserver {
    void onPlayerInventoryChanged(Player p);
}

// 2. Il Subject (Chi viene osservato)
public class Player {
    private List<PlayerObserver> observers = new ArrayList<>();
    private int diamonds = 0;

    public void addObserver(PlayerObserver o) {
        observers.add(o);
    }

    public void addDiamond() {
        this.diamonds++;
        notifyObservers(); // Notifica tutti quando lo stato cambia!
    }

    private void notifyObservers() {
        for (PlayerObserver o : observers) {
            o.onPlayerInventoryChanged(this); // Passa se stesso come contesto
        }
    }

    public int getDiamonds() { return this.diamonds; }
}

// 3. Un Observer concreto
public class AchievementSystem implements PlayerObserver {
    @Override
    public void onPlayerInventoryChanged(Player p) {
        if (p.getDiamonds() == 1) {
            System.out.println("ACHIEVEMENT UNLOCKED: DIAMONDS!");
        }
    }
}
```

Commento: Se in futuro vorremo aggiungere un sistema grafico che aggiorna l'icona dell'inventario, basterà creare una `GUIObserver` e aggiungerla al `Player`. Il codice del `Player` non dovrà subire alcuna modifica.

18.2.2 Il Visitor Pattern (Entity e DamageVisitor)

```
// 1. Interfaccia Visitor (L'operazione esterna)
public interface EntityVisitor {
    void visit(Zombie z);
    void visit(Creeper c);
}

// 2. Interfaccia Element (Chi accetta la visita)
public interface Entity {
    void accept(EntityVisitor visitor);
}

// 3. Elementi concreti (La gerarchia stabile)
public class Zombie implements Entity {
    @Override
    public void accept(EntityVisitor visitor) {
        visitor.visit(this); // Risoluzione del tipo esatto a runtime
    }
}

public class Creeper implements Entity {
    @Override
    public void accept(EntityVisitor visitor) {
        visitor.visit(this);
    }
}

// 4. Visitor concreto (La nuova operazione)
public class HolyDamageVisitor implements EntityVisitor {
    @Override
    public void visit(Zombie z) {
        System.out.println("Holy Damage vs Zombie: CRITICAL HIT! (x2 Damage)");
    }

    @Override
    public void visit(Creeper c) {
        System.out.println("Holy Damage vs Creeper: Normal Hit.");
    }
}
```

Commento: Senza il *Visitor*, per implementare il "danno sacro" avremmo dovuto usare bruttissimi if (e instanceof *Zombie*) o sporcare le classi originali aggiungendo `takeHolyDamage()`. Con il *Visitor*, ogni nuova logica complessa risiede nella propria classe separata.

18.3 Collegamento con i tutorati e sviluppi futuri

Anche se il **Tutorato 4 e 5** non chiedono esplicitamente di implementare un *Visitor*, introducono le fondamenta per l'architettura dei successivi laboratori. L'*Observer Pattern*, in particolare, è il ponte concettuale che ci guiderà dritti alla **Lezione 22**: il pattern **Model-View-Controller (MVC)**. Nel MVC, la *View* (l'interfaccia grafica) si comporta come un *Observer* in attesa che il *Model* (i dati di gioco) la notifichi di eventuali cambiamenti, separando in modo perfetto la logica dalla grafica.

19 Lezione 19 (Lab 4): Esercitazione su Collections e Pattern

19.1 Spiegazione teorica e Struttura del Laboratorio

Il quarto laboratorio è un banco di prova per verificare la comprensione delle lezioni 17 e 18. Viene richiesto di gestire un sistema che non ha un solo tipo di dato e un solo comportamento, ma una gerarchia complessa in cui gli oggetti devono essere raggruppati, filtrati, ordinati e manipolati da agenti esterni.

Le fasi del laboratorio si concentrano su:

- **Gestione Avanzata delle Liste:** Non basta più fare una semplice `List<Entity>`. Si richiede di effettuare ricerche (es. trovare l'elemento con il costo maggiore), calcoli aggregati (es. la somma totale degli abbonamenti) e ordinamenti personalizzati tramite `Comparator`.
- **Separazione delle responsabilità (Pattern):** Quando una logica esterna deve agire su un oggetto (es. un Tecnico che ripara un Dispositivo, o uno Sconto che si applica a un Abbonamento), non bisogna "sporcare" la classe del modello dati inserendovi la logica dell'operatore. Si sfruttano interfacce e pattern (come lo `Strategy` o l'embrione del `Visitor`).

19.2 Implementazione pratica: Collections e Ordinamento

Prendendo ispirazione dall'esercizio della gestione degli Abbonati, vediamo come strutturare il modello e l'ordinamento.

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

// 1. Classe Base Astratta che implementa Comparable (Ordinamento Naturale)
public abstract class Abbonato implements Comparable<Abbonato> {
    protected String nome;
    protected double costoBase;

    public Abbonato(String nome, double costoBase) {
        this.nome = nome;
        this.costoBase = costoBase;
    }

    public abstract double getCostoFinale();
    public String getNome() { return this.nome; }

    // Ordinamento naturale di default: per nome (alfabetico)
    @Override
    public int compareTo(Abbonato other) {
        return this.nome.compareTo(other.nome);
    }
}

// 2. Sottoclasse specifica
public class Studente extends Abbonato {
    public Studente(String nome) { super(nome, 50.0); }

    @Override
    public double getCostoFinale() {
        return this.costoBase * 0.8; // 20% di sconto studenti
    }
}
```

Per ordinare dinamicamente gli abbonati non per nome, ma per *costo*, si crea un `Comparator` esterno:

```
public class GymManager {
    private List<Abbonato> iscritti = new ArrayList<>();

    public void aggiungi(Abbonato a) { iscritti.add(a); }

    public void stampaOrdinatiPerCosto() {
        // Uso di un Comparator con classe anonima (o Lambda in Java 8+)
        Collections.sort(iscritti, new Comparator<Abbonato>() {
            @Override
            public int compare(Abbonato a1, Abbonato a2) {
                return Double.compare(a1.getCostoFinale(), a2.getCostoFinale());
            }
        });

        for (Abbonato a : iscritti) {
            System.out.println(a.getNome() + " - " + a.getCostoFinale());
        }
    }
}
```

19.3 Esercizi e approfondimenti dal tutorato (Lab 4): Polimorfismo e Collezioni

Nei laboratori intermedi come il **Tutorato 4**, viene valutata la capacità dello studente di far interagire collezioni generiche (`List<T>`) con il polimorfismo, elaborando dati aggregati senza ricorrere a cast espliciti o a codice procedurale.

19.3.1 Esercizio: Palestra, Atleti ed Enum Avanzate

Completiamo l'esempio precedente della palestra introducendo la classe `Atleta`, il cui costo dell'abbonamento varia dinamicamente in base alla `Categoria` di appartenenza. Modelliamo la categoria tramite un'Enum avanzata.

```
// 1. Enum con campi interni (Design pattern molto richiesto all'esame)
public enum Categoria {
    PRINCIPIANTE(0.0),
    INTERMEDIO(15.0),
    AGONISTA(30.0);

    private final double sovrapprezzo;

    // Il costruttore di una Enum è implicitamente privato
    Categoria(double sovrapprezzo) {
        this.sovrapprezzo = sovrapprezzo;
    }

    public double getSovrapprezzo() { return sovrapprezzo; }
}

// 2. Sottoclasse che sfrutta l'Enum per il calcolo polimorfico
public class Atleta extends Abbonato {
    private final Categoria categoria;

    public Atleta(String nome, Categoria categoria) {
        super(nome, 60.0); // Costo base maggiore per gli atleti
        this.categoria = categoria;
    }

    @Override
    public double getCostoFinale() {
        // Il costo cambia dinamicamente estraendo il dato dalla Enum
        return this.costoBase + categoria.getSovrapprezzo();
    }
}
```

19.3.2 Gestione Avanzata della Collezione

All'interno del manager, calcoliamo il bilancio totale della palestra. Grazie al *Dynamic Dispatch*, la lista può contenere sia *Studente* che *Atleta*, ma il ciclo non avrà mai bisogno di verificare l'identità dell'oggetto.

```
public class GymManager {
    private List<Abbonato> iscritti = new ArrayList<>();

    public void aggiungi(Abbonato a) { iscritti.add(a); }

    // Calcolo aggregato su una collezione polimorfica
    public double calcolaIncassoTotale() {
        double totale = 0.0;

        for (Abbonato a : iscritti) {
            // A runtime, Java sceglierà automaticamente se applicare lo
            // sconto (Studente) o aggiungere il sovrapprezzo (Atleta)
            totale += a.getCostoFinale();
        }

        return totale;
    }
}
```

Analisi didattica per l'esame:

- **Enum non banali:** Usare una *Enum* non solo come semplice "etichetta", ma come contenitore di logica e parametri costanti (il sovrapprezzo) è una *best practice*. Questa architettura elimina totalmente la necessità di scrivere un `switch(categoria)` all'interno del metodo `getCostoFinale()`.
- **Potenza del Polimorfismo:** Il metodo `calcolaIncassoTotale()` dimostra perché l'OOP è superiore al procedurale in architetture complesse. La classe `GymManager` si disinteressa completamente di quanti e quali tipi di abbonati esistano nel sistema; finché rispettano il contratto di `Abbonato`, il bilancio aziendale si calcolerà senza bug.

20 Lezione 20: Programmazione Funzionale e Fondamenti di JavaFX

20.1 Programmazione Funzionale (Interfacce Funzionali e Lambda)

Fino a Java 7, l'unico modo per passare un "comportamento" come parametro a un metodo era creare un oggetto (spesso tramite una *Classe Anonima*) che implementasse una specifica interfaccia. Questo portava a scrivere molto codice ripetitivo (boilerplate). Con Java 8, è stato introdotto il paradigma funzionale per rendere il codice più conciso, leggibile e orientato al *cosa* fare piuttosto che al *come* farlo (approccio dichiarativo).

- **Interfacce Funzionali (SAM - Single Abstract Method):** Un'interfaccia si definisce "funzionale" se contiene **esattamente un solo metodo astratto** (può comunque contenere infiniti metodi `default` o `static`).
 - L'annotazione opzionale `@FunctionalInterface` dice al compilatore di verificare che la regola del singolo metodo sia rispettata.
 - Interfacce classiche come `Runnable` (ha solo `run()`) o `Comparator` (ha solo `compare()`) sono interfacce funzionali.
- **Espressioni Lambda (->):** Sono "funzioni anonime" che permettono di implementare al volo il singolo metodo di un'interfaccia funzionale, senza dover dichiarare una classe intera.
 - **Sintassi base:** `(parametri) -> { corpo della funzione }`
 - Se c'è un solo parametro, le parentesi tonde si possono omettere: `p -> { ... }`
 - Se il corpo ha una sola istruzione, le parentesi graffe e il `return` si possono omettere: `(a, b) -> a + b`
- **Interfacce Funzionali di Sistema (`java.util.function`):** Java fornisce interfacce standard pronte all'uso per non doverne creare di nuove ogni volta:
 - `Predicate<T>`: Prende in input un `T` e restituisce un `boolean` (utile per filtrare).
 - `Consumer<T>`: Prende in input un `T` e non restituisce nulla (utile per stampare o modificare).
 - `Function<T, R>`: Prende un `T` e restituisce un `R` (utile per mappare/trasformare).
- **Method References (::):** Se una lambda fa solo una cosa, ovvero richiamare un metodo già esistente, si può usare una sintassi ancora più compatta: `Classe::metodo` (es. `System.out::println`).

20.1.1 Refactoring: Dalle Classi Anonime alle Lambda

Riprendiamo l'esempio del `Comparator` della Lezione 19 e vediamo come si evolve.

```
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

public class LambdaRefactoring {
    public void sortPlayers(List<Player> players) {
        // APPROCCIO VECCHIO (Java 7): Classe Anonima
        Collections.sort(players, new Comparator<Player>() {
            @Override
            public int compare(Player p1, Player p2) {
                return Integer.compare(p1.getScore(), p2.getScore());
            }
        });

        // NUOVO APPROCCIO (Java 8+): Espressione Lambda
        // Il compilatore "capisce" che p1 e p2 sono di tipo Player
        Collections.sort(players, (p1, p2) -> Integer.compare(p1.getScore(),
                                                                p2.getScore()));

        // APPROCCIO MODERNO CON METHOD REFERENCE (Ottimale)
        players.sort(Comparator.comparingInt(Player::getScore));
    }
}
```


20.1.2 Uso pratico di Predicate e Consumer

Vediamo come passare un blocco di codice come parametro per filtrare un inventario.

```
import java.util.ArrayList;
import java.util.List;
import java.util.function.Predicate;
import java.util.function.Consumer;

public class Inventory {
    private List<Item> items = new ArrayList<>();

    // Questo metodo accetta una CONDIZIONE e un'AZIONE
    public void processItems(Predicate<Item> condition,
                           Consumer<Item> action) {
        for (Item item : items) {
            // Se l'oggetto rispetta la condizione...
            if (condition.test(item)) {
                // ...esegui l'azione
                action.accept(item);
            }
        }
    }

    public static void testLambda() {
        Inventory inv = new Inventory();

        // Uso le lambda per definire la logica "al volo"
        // Condizione: l'arma è rotta (durability == 0)
        // Azione: stampala a schermo
        inv.processItems(
            item -> item.getDurability() == 0,
            item -> System.out.println("Item rotto: " + item.getName())
        );
    }
}
```

20.2 Fondamenti di JavaFX: Interfacce Grafiche (GUI)

La seconda parte della lezione introduce **JavaFX**, il framework standard moderno per lo sviluppo di interfacce grafiche in Java. Per non mescolare la logica del programma con la grafica, è fondamentale capire come JavaFX struttura lo schermo.

- **Stage e Scene:** Lo **Stage** rappresenta la finestra fisica creata dal sistema operativo (quella con la cornice e i tasti chiudi/minimizza). La **Scene** è il contenuto interno della finestra. Uno **Stage** può mostrare una sola **Scene** alla volta.
- **Lo Scene Graph e i Nodi (Nodes):** Tutto ciò che viene visualizzato fa parte di un albero gerarchico chiamato *Scene Graph*. Ogni elemento visivo è un **Node**.
- **Text vs Label:** Sebbene entrambi mostrino stringhe, **Text** è un nodo grafico primitivo (usato per disegnare testo come fosse una forma geometrica), mentre **Label** è un elemento di UI strutturato (Control) a cui si possono applicare margini, sfondi e stili.

20.2.1 I Layout Panes (Contenitori)

Per posizionare i Nodi non si usano coordinate assolute (X, Y) che si romperebbero se l'utente ridimensionasse la finestra. Si usano invece dei contenitori invisibili chiamati *Panes* che impaginano automaticamente i propri figli.

- **HBox (Horizontal Box):** Dispone i nodi affiancati in una singola riga orizzontale.
- **VBox (Vertical Box):** Dispone i nodi impilati in una singola colonna verticale.
- **BorderPane:** Divide lo schermo in 5 aree logiche (**Top, Bottom, Left, Right, Center**). È il layout principale per strutturare l'impalcatura di un'app.
- **StackPane:** Impila i nodi uno sopra l'altro lungo l'asse Z (la profondità). Il primo nodo inserito diventa lo sfondo, gli ultimi coprono i precedenti.

20.2.2 Costruire la finestra di base

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;

public class BasicJavaFX extends Application {
    @Override
    public void start(Stage primaryStage) {
        // 1. Creazione del nodo visivo
        Label lblMessage = new Label("Benvenuto in JavaFX!");

        // 2. Creazione del Layout (Contenitore)
        VBox root = new VBox();
        root.getChildren().add(lblMessage); // Aggiunge il nodo al contenitore

        // 3. Creazione della Scene e inserimento nello Stage
        Scene scene = new Scene(root, 400, 300);
        primaryStage.setTitle("Finestra Principale");
        primaryStage.setScene(scene);
        primaryStage.show(); // Rende visibile la finestra
    }
}
```

20.3 Esercizi e approfondimenti dal tutorato: Unire Lambda e JavaFX

Nel corso dei laboratori, l'introduzione della programmazione funzionale non è fine a se stessa, ma è il prerequisito fondamentale per gestire le interfacce grafiche in modo pulito e moderno. Nel **Tutorato 5**, l'uso delle espressioni Lambda diventa lo standard assoluto per gestire le azioni dell'utente (come i click) e per manipolare i dati da mostrare a schermo.

20.3.1 Esercizio: Filtro dinamico con Interfaccia Grafica

Immaginiamo una semplice applicazione che possiede un inventario e un bottone. Cliccando il bottone, l'applicazione usa l'approccio funzionale per filtrare l'inventario e aggiorna il testo della finestra. Questo unisce perfettamente i due argomenti principali della Lezione 20.

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.control.Label;
import javafx.scene.layout.VBox;
import javafx.stage.Stage;
import java.util.Arrays;
import java.util.List;
```

```

import java.util.stream.Collectors;

public class LambdaFXTest extends Application {
    private List<String> inventario = Arrays.asList("Spada", "Arco", "Scudo", "Pozione");

    @Override
    public void start(Stage primaryStage) {
        Label lblRisultato = new Label("Oggetti: " + inventario);
        Button btnFiltra = new Button("Mostra solo armi con la 'S'");

        // 1. USO DELLA LAMBDA IN JAVA FX: Gestione dell'evento di click
        // SetOnAction richiede un'interfaccia funzionale EventHandler
        btnFiltra.setOnAction(event -> {

            // 2. USO DELLA LAMBDA SUI DATI: Filtro tramite Predicate e Stream
            String filtrati = inventario.stream()
                .filter(item -> item.startsWith("S")) // Il Predicate!
                .collect(Collectors.joining(", "));

            lblRisultato.setText("Filtrati: " + filtrati);
        });

        // Setup del Layout (VBox) e della Scena
        VBox root = new VBox(10); // 10 pixel di spaziatura tra i nodi
        root.getChildren().addAll(btnFiltra, lblRisultato);

        Scene scene = new Scene(root, 300, 150);
        primaryStage.setTitle("Test Lambda & JavaFX");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
}

```

Analisi didattica per l'esame:

- **Sintassi compatta e Leggibilità:** Prima di Java 8, per far funzionare quel bottone avremmo dovuto istanziare una pesante classe anonima (`new EventHandler<ActionEvent>() { ... }`). La sintassi `event -> { ... }` riduce drasticamente il "rumore visivo" e ci permette di concentrarci solo sulla logica di business.
- **Dichiarativo vs Imperativo:** Il filtraggio tramite `stream().filter(...)` dimostra la potenza dell'approccio dichiarativo che viene spesso premiato all'esame. Invece di istanziare una nuova lista vuota, fare un ciclo `for`, fare un `if` e aggiungere gli elementi uno a uno, dichiariamo semplicemente al programma *cosa* vogliamo (elementi che iniziano con "S").

21 Lezione 21: Gestione degli Eventi in JavaFX (Event Handling)

21.1 Spiegazione teorica

Ora che sappiamo come costruire un'interfaccia statica tramite Nodi e Layout (Lezione 20), dobbiamo renderla interattiva. In JavaFX, le interazioni dell'utente (click del mouse, pressione di tasti, trascinamenti) generano degli oggetti chiamati **Eventi**. La comprensione del ciclo di vita di un evento è un requisito fondamentale per superare l'esame e per implementare correttamente la futura architettura MVC.

Quando si verifica un evento (es. un click su un bottone), JavaFX determina qual è il nodo bersaglio (*Target Node*) e costruisce un percorso logico (*Event Dispatch Chain*) dalla radice della scena fino al bersaglio. L'evento viaggia su questo percorso in due fasi distinte:

- **Fase 1: Cattura (Event Filters):** L'evento viaggia dall'alto verso il basso (*Top-Down*), partendo dallo **Stage**, attraversando la **Scene** e scendendo attraverso i layout padri (es. **VBox**) fino ad arrivare al nodo bersaglio. In questa fase, l'evento può essere intercettato registrando un **Event Filter** tramite il metodo `addEventFilter()`. Serve per intercettare (o bloccare) gli eventi prima che raggiungano i figli.
- **Fase 2: Risalita o Emersione (Event Handlers):** Una volta raggiunto il nodo bersaglio, l'evento fa il percorso inverso dal basso verso l'alto (*Bottom-Up*), tornando dal bersaglio fino alla radice. In questa fase, l'evento viene gestito dai normali **Event Handlers** registrati tramite il metodo `addEventHandler()` (o tramite metodi scorciatoia come `setOnAction()`).
- **Consumare un evento (`event.consume()`):** In qualsiasi momento, durante la fase di cattura o di emersione, un filtro o un handler può invocare il metodo `event.consume()`. Questo comando "distrugge" l'evento, interrompendo immediatamente la catena di propagazione. I nodi successivi nella catena non riceveranno mai quell'evento.

21.2 Implementazione del codice

Vediamo un esempio didattico in cui un bottone si trova all'interno di un contenitore VBox. Registriamo filtri e handler su entrambi per osservare il rigoroso ordine di esecuzione.

```
import javafx.scene.layout.VBox;
import javafx.scene.control.Button;
import javafx.scene.input.MouseEvent;

public class EventHandlingExample {
    public void setupEvents(VBox root, Button myButton) {

        // 1. EVENT FILTER sul contenitore padre (Fase di Cattura - Top Down)
        root.addEventFilter(MouseEvent.MOUSE_CLICKED, event -> {
            System.out.println("1. VBox Filter intercetta (Scendendo)");
        });

        // 2. EVENT FILTER sul nodo bersaglio
        myButton.addEventFilter(MouseEvent.MOUSE_CLICKED, event -> {
            System.out.println("2. Button Filter intercetta il click");
        });

        // 3. EVENT HANDLER sul nodo bersaglio (Fase Emersione - Bottom Up)
        myButton.addEventHandler(MouseEvent.MOUSE_CLICKED, event -> {
            System.out.println("3. Button Handler gestisce il click");

            // Consumiamo l'evento! Non risalirà più verso i padri.
            event.consume();
        });

        // 4. EVENT HANDLER sul contenitore padre
        root.addEventHandler(MouseEvent.MOUSE_CLICKED, event -> {
            // QUESTO CODICE NON VERRÀ MAI ESEGUITO perché
            // il Button Handler (al punto 3) ha consumato l'evento.
            System.out.println("4. VBox Handler (Risalendo)");
        });
    }
}
```

Analisi del flusso: Quando l'utente clicca il `myButton`, la console stamperà i passaggi 1, 2 e 3. L'uso congiunto della teoria vista nella Lezione 20 (le *Lambda Expressions*) rende la registrazione di questi eventi molto compatta e leggibile: si passa al nodo direttamente la funzione da eseguire `event -> { ... }`.

21.3 Collegamento con il pattern MVC

Perché è così importante padroneggiare la gestione degli eventi prima di affrontare l'MVC?

Nel **Model-View-Controller**, la Vista (*View*) non deve contenere alcuna logica applicativa. Il suo unico scopo è mostrare elementi visivi e "ascoltare" le interazioni dell'utente. L'architettura corretta (che vedremo nella prossima lezione) prevede che la *View* utilizzi il metodo `addEventHandler` sui suoi elementi grafici, e che all'interno della Lambda expression si limiti a invocare un metodo del *Controller*. L'evento di JavaFX funge quindi da "ponte" e innesco per la vera logica di business.

22 Lezione 22: Architettura del Software (Pattern Model-View-Controller)

22.1 Spiegazione teorica

L'apice del "programming in the large" orientato agli oggetti si raggiunge con la creazione di interfacce grafiche (GUI). Quando si progetta un'applicazione interattiva, il rischio maggiore è creare "classi dio" (God Classes) che contengono contemporaneamente la logica del programma e il codice per disegnare i bottoni sullo schermo. Questo rende il codice impossibile da testare, mantenere o riutilizzare.

Per risolvere questo problema, l'industria software adotta il pattern architetturale **Model-View-Controller (MVC)**, che impone una rigida separazione delle responsabilità (Separation of Concerns):

[Image of MVC Model View Controller architecture diagram]

- **Model (Il Modello):** Contiene i dati puri, lo stato dell'applicazione e le regole di business (es. la classe `Player`, l'inventario, la vita). **Non sa assolutamente nulla dell'interfaccia grafica.** Non importa `javafx.scene.*`. Se i suoi dati cambiano, notifica gli interessati (spesso usando l'*Observer Pattern*).
- **View (La Vista):** È la rappresentazione visiva (i bottoni, i testi, i layout visti nella Lezione 20). Il suo unico compito è mostrare i dati del Modello all'utente. **Non contiene alcuna logica di business** (non decide se un giocatore muore, si limita a disegnare la barra della vita vuota).
- **Controller (Il Controllore):** È il "direttore d'orchestra". Registra gli *Event Handlers* sulla Vista (come visto nella Lezione 21) e gestisce l'input dell'utente. Quando l'utente clicca, il Controller riceve l'evento e invoca i metodi appropriati per modificare il Modello.

22.2 Implementazione Base: multiInputsMVC

A lezione è stato illustrato l'esempio didattico `multiInputsMVC`, in cui la Vista delega le azioni al Controller tramite eventi e lambda expressions.

22.2.1 La Vista (Intercettare l'input visivo)

```
package lecture22.multiInputsMVC.view;

import javafx.scene.control.Button;
import javafx.scene.input.MouseEvent;
import javafx.scene.input.KeyCode;
import javafx.scene.input.KeyEvent;

public class MultiStoneBlockView {
    private Button nameButton = new Button("Mine Block");

    // Il costruttore o un metodo di inizializzazione riceve il Controller
    public void setupEvents(MultiStoneBlockController mc) {

        // Uso delle Lambda per collegare l'evento UI al Controller
        this.nameButton.addEventHandler(MouseEvent.MOUSE_CLICKED, event -> {
            mc.logic(); // La Vista non sa COSA succede, avvisa solo il Controller
        });

        this.nameButton.addEventHandler(KeyEvent.KEY_PRESSED, event -> {
            if (event.getCode() == KeyCode.N) {
                mc.logic();
            }
        });
    }
}
```

22.2.2 Il Controller (La logica di smistamento)

```
package lecture22.multiInputsMVC.controller;

public class MultiStoneBlockController {
    private Model model;
    private MultiStoneBlockView view;

    public MultiStoneBlockController(Model model, MultiStoneBlockView view) {
        this.model = model;
        this.view = view;
        this.view.setupEvents(this); // Collega gli eventi alla View
    }

    // Metodo invocato dalla Vista quando accade un evento
    public void logic() {
        try {
            // Il Controller altera il Modello in modo sicuro
            model.mine();
        } catch (BlockAlreadyBrokenException e) {
            // Il Controller gestisce gli errori e comanda alla Vista
            // di mostrare un popup
            view.showError("Il blocco è già rotto!");
        }
    }
}
```

22.3 MVC Avanzato: Gestione delle Collezioni (collectionsMVC)

Cosa succede quando non abbiamo un solo oggetto, ma una lista intera (es. uno stack di blocchi o un inventario)? L'architettura deve scalare. Nel repository del corso, il professore mostra che **l'architettura dei Controller deve rispecchiare quella del Model**.

Se il `BlockStackModel` contiene una `List<StoneBlock>`, allora il `MainController` non gestirà direttamente i blocchi, ma conterrà una `List<StoneBlockController>` (dei "sotto-controller").

22.3.1 L'Ordinamento dei Controller (Il trucco d'esame)

Per ordinare graficamente la Vista (es. in base al nome dei blocchi), si sfrutta un meccanismo di "Doppio Ordinamento" estremamente pulito:

1. Si crea un **Comparator** per i **Controller** (es. `BlockControllerComparatorByName`).
2. Questo comparatore estrae i modelli dai due controller e li confronta delegando l'operazione al **Comparator** del Modello.
3. Il Main Controller ordina la propria lista di sotto-controller.
4. Infine, il Main Controller svuota la Vista principale e la ripopola nell'ordine corretto, estraendo le singole sotto-viste dai controller appena ordinati.

```
// Esempio logico di Ordinamento dei Controller (Dentro al Main Controller)
public void sortAlphabetically() {
    // 1. Ordina la lista dei Controller delegando al Model
    this.blockControllers.sort((c1, c2) -> {
        return c1.getModel().getName().compareTo(c2.getModel().getName());
    });

    // 2. Comanda alla View principale di svuotarsi
    this.mainView.clearAll();

    // 3. Ripopola la View nell'ordine corretto
    for (StoneBlockController c : blockControllers) {
        this.mainView.addBlockView(c.getView());
    }
}
```

Commento architetturale: Questo pattern garantisce che la Vista principale non debba mai scorrere direttamente i modelli, mantenendo il disaccoppiamento totale richiesto all'esame per prendere il massimo dei voti.

23 Lezione 23 (Lab 5): Esercitazione su Layout grafici e implementazione MVC

23.1 Spiegazione teorica e Struttura del Laboratorio

Il quinto laboratorio è incentrato sull'applicazione rigorosa del pattern Model-View-Controller (MVC) integrato con l'interfaccia grafica JavaFX. A differenza della pura teoria vista in precedenza, qui l'obiettivo è comprendere come far scalare l'architettura passando da esempi didattici a veri e propri progetti organizzati su più file e package.

Il laboratorio è strutturato in tre fasi di analisi e sviluppo:

- **Fase 1: Che layout usare:** Si discute su come organizzare visivamente gli elementi usando i contenitori di JavaFX (`HBox`, `VBox`, `BorderPane`, ecc.) per disporre pulsanti, testi e form di input in modo che siano reattivi al ridimensionamento della finestra.
- **Fase 2: Come fare MVC semplice:** Creazione dello scheletro base in cui un singolo Controller gestisce le interazioni tra un Model semplice e una singola View.
- **Fase 3: Come fare MVC complesso:** Estensione del sistema per gestire modelli ricchi (gerarchie di classi, composizione) ed eventi ramificati su più viste.

23.2 Architettura dei Package (Il "Segreto" per l'Esame)

La corretta suddivisione del progetto in directory (package) è il primo fondamentale criterio di valutazione all'esame pratico. Nel tutorato relativo (es. sistema di abbonamenti a ventilatori) la struttura ideale si presenta così:

```
src/
├── model/                                // IL MODELLO (Logica e Dati Puri)
│   ├── alimentazioni/                    // -> Sottosistema compositivo
│   │   ├── AbstractAlimentazione.java
│   │   ├── AlimentazioneBatteria.java
│   │   └── AlimentazionePresa.java
│   ├── enums/                           // -> Costanti di dominio
│   │   └── Marche.java
│   ├── fans/                             // -> Gerarchia degli oggetti principali
│   │   ├── AbstractFan.java
│   │   ├── PiantanaFan.java
│   │   └── SoffittoFan.java
│   ├── Utente.java                       // -> Stato dell'attore principale
│   └── Vetrina.java                      // -> Il contenitore centrale (Manager)
├── view/                                 // LA VISTA (Solo elementi JavaFX)
│   └── MainView.java
├── controller/                           // IL CONTROLLLORE (Ponte e logica eventi)
│   └── SubscriptionController.java
└── Main.java                             // Entry point dell'applicazione
```

Analisi del Flusso MVC Complesso:

1. **Inizializzazione (Main.java):** All'avvio, il programma non deve contenere logica. Si limita a istanziare la *Vetrina* (Model) e la *MainView* (View), passandole come argomenti al costruttore del *SubscriptionController*.
2. **Visualizzazione (MainView):** La vista non legge direttamente gli oggetti dal modello per stamparli. L'accoppiamento deve essere debole. La vista delega al controller la richiesta dei dati da mostrare e costruisce lo *Scene Graph*.
3. **Interazione (SubscriptionController):** Quando un *Utente* decide di compiere un'azione (es. clicca su "Abbonati"), la *View* cattura l'evento del mouse e invoca un metodo sul *Controller*. Il *Controller* invoca a sua volta i metodi del *Model* (es. la classe *Vetrina*).
4. **Gestione Eccezioni:** Se la *Vetrina* lancia un'eccezione (es. "Modello esaurito" o "Credito insufficiente"), il *Controller* la cattura nel blocco `try-catch` e comanda alla *View* di renderizzare un *Alert* (messaggio d'errore) per l'utente.

Questo schema garantisce che, se domani volessimo cambiare la grafica, l'intera cartella `model` e gran parte del `controller` non richiederebbero alcuna modifica, realizzando in pieno il principio dell'OOP.

24 Lezione 24 (Lab 6): Conclusione ed Esercitazione Finale

24.1 Spiegazione teorica e Struttura della Simulazione

L'ultimo laboratorio del corso non introduce nuovi costrutti sintattici, ma rappresenta la prova generale prima dell'esame (Risoluzione di un tema proposto). L'obiettivo è dimostrare di saper orchestrare tutti i concetti visti finora (*Programming in the Large*) all'interno di un'unica architettura robusta.

Un tema d'esame tipico richiede di applicare simultaneamente:

- **Architettura MVC:** Separazione rigorosa tra chi detiene i dati (*Model*), chi li mostra (*View*) e chi gestisce le interazioni (*Controller*).
- **Ereditarietà e Polimorfismo:** Costruzione di gerarchie profonde sfruttando il *Dynamic Binding* per evitare cast e controlli di tipo a runtime.
- **Design Patterns:** Uso del *Template Method* per definire scheletri algoritmici, dello *Strategy* o del *Visitor* per operazioni esterne, e di *Factory* per la creazione degli oggetti.
- **Robustezza (Eccezioni e Invarianti):** Uso di eccezioni custom (*Checked* e *Unchecked*) per bloccare transizioni di stato non valide e garantire che gli oggetti non assumano mai stati illogici.
- **Collezioni e Generics:** Uso avanzato di Liste, abbinate a *Comparable* e *Comparator* per l'ordinamento.

24.2 Caso Studio Riassuntivo: Il Sistema QuestLog

L'esercizio basato sulla gestione delle missioni di **Geraldo** è il prototipo perfetto di un esame da 30 e lode. Analizziamo come i vari pezzi del puzzle si incastrano.

24.2.1 1. Il Modello: Ereditarietà e Invarianti

Si parte definendo le interfacce e le classi astratte. Una **AbstractQuest** definisce i campi base (titolo, ricompensa) e delega alle sottoclassi i requisiti specifici.

```
package model.quests;
import model.exceptions.*;

public abstract class AbstractQuest implements Quest {
    private final String title;
    private boolean isCompleted = false;

    public AbstractQuest(String title) {
        this.title = title;
    }

    // Il metodo lancia eccezioni custom se le regole di business sono violate
    public void completeQuest(int playerLevel) throws QuestException {
        if (isCompleted) {
            throw new AlreadyCompletedException("Missione già completata!");
        }
        if (!checkRequirements(playerLevel)) {
            throw new InsufficientLevelException("Livello troppo basso.");
        }
        this.isCompleted = true;
    }

    // Hook polimorfico (Template Method) che le sottoclassi devono implementare
    protected abstract boolean checkRequirements(int playerLevel);
}
```

24.2.2 2. Il Modello: Gestione delle Collezioni

La classe `QuestLog` fa da "Database" (Manager) incapsulando una `List<Quest>` e nascondendo i dettagli implementativi all'esterno.

```
package model;
import model.quests.Quest;
import java.util.ArrayList;
import java.util.List;

public class QuestLog {
    private List<Quest> activeQuests = new ArrayList<>();

    public void addQuest(Quest q) {
        // L'operatore equals() deve essere stato sovrascritto in Quest!
        if (!activeQuests.contains(q)) {
            activeQuests.add(q);
        }
    }
}
```

24.2.3 3. Il Controller: Il Direttore d'Orchestra

Il `QuestController` intercetta le richieste dell'utente (provenienti dalla *View*), invoca i metodi del *Model* e gestisce eventuali problemi in modo "pulito", catturando le eccezioni.

```
package controller;
import model.Geraldo;
import model.exceptions.QuestException;
import view.MainView;

public class QuestController {
    private Geraldo player;
    private MainView view;

    public QuestController(Geraldo player, MainView view) {
        this.player = player;
        this.view = view;
    }

    public void attemptQuestCompletion(int questIndex) {
        try {
            // Delega la logica al modello
            player.getLog().completeQuestAt(questIndex, player.getLevel());
            view.showMessage("Successo! Hai completato la missione.");
        } catch (QuestException e) {
            // Cattura l'errore di dominio e aggiorna la UI
            view.showError("Fallimento: " + e.getMessage());
        }
    }
}
```

24.3 Conclusione e Consigli per l'Esame

1. **Leggere prima i Sostantivi e i Verbi:** Durante la lettura del testo d'esame, cerciate i sostantivi (diventeranno `Classi` o `Enum`) e i verbi (diventeranno `Metodi`).
2. **Disegnare l'albero delle directory:** Prima di scrivere una singola riga di codice, create i package `model`, `view` e `controller` nell'IDE.
3. **Incapsulamento totale:** Tutti i campi devono essere `private`. Usate `final` ovunque sia possibile per le costanti. Fornite i `getter` solo se strettamente necessari per la View. Non fornite `setter` indiscriminatamente se l'oggetto deve mutare tramite metodi di business (es. `damage()` e non `setHealth()`).
4. **Nessun `if` sui tipi:** Se vi trovate a scrivere `if (obj instanceof T)` oppure a fare `(T) obj`, fermatevi. State sbagliando l'architettura. Spostate quel comportamento in un metodo polimorfico o usate un *Visitor Pattern*.

25 Domande d'Esame: Vero o Falso

Di seguito sono riportate alcune domande classiche da esame, con relativa soluzione e spiegazione dettagliata, supportata rigorosamente dai concetti architetturali visti a lezione.

- **Domanda 1**

Testo: In Java posso fare overloading di un metodo cambiando il tipo di ritorno.

Risposta esatta: Falso

Spiegazione: L'overloading (sovraccarico) si ottiene quando si definiscono più metodi nella stessa classe con lo stesso nome ma con parametri diversi, sia per tipo che per numero. Cambiare solamente il tipo di ritorno senza alterare i parametri non è sufficiente e genererà un errore a livello di compilazione.

- **Domanda 2**

Testo: In Java non si può decidere quando viene invocato il garbage collector.

Risposta esatta: Vero

Spiegazione: Nel linguaggio Java la gestione della memoria Heap avviene tramite la Garbage Collection, dove è il sistema stesso a pulire in automatico la memoria che non risulta più raggiungibile. Il programmatore non può deciderne o forzarne l'esecuzione in modo deterministico.

- **Domanda 3**

Testo: La seguente inizializzazione è corretta:

```
ArrayList<Integer> i = new ArrayList<>();
```

Risposta esatta: Vero

Spiegazione: Questa sintassi è corretta. Negli appunti vengono mostrati esempi simili per l'inizializzazione di liste dinamiche. L'operatore diamante <> permette al compilatore di dedurre automaticamente il tipo generico.

- **Domanda 4**

Testo: Data una classe C con un campo `private int c` e con una inner class D, dentro a un metodo di D possiamo fare `int i = new C().c;`

Risposta esatta: Vero

Spiegazione: Il modificatore `private` rende un campo visibile all'interno della classe stessa. Poiché una inner class (classe interna) è dichiarata e risiede fisicamente all'interno della classe genitore, ad essa è consentito accedere ai membri privati della classe che la contiene.

- **Domanda 5**

Testo: Dato un package P, data una classe C dentro P con campo `protected int c`, data una classe D dentro a P, dentro a un metodo di D possiamo fare `int i = new C().c;`

Risposta esatta: Vero

Spiegazione: Il modificatore `protected` rende i campi e i metodi visibili a tutte le classi che risiedono all'interno dello stesso package, oltre che alle sottoclassi. Essendo C e D nello stesso package P, la visibilità è garantita.

- **Domanda 6**

Testo: Data una classe `D` che `extends Exception` (in Java), possiamo fare `throw new D()` dentro a un blocco `catch`.

Risposta esatta: Vero

Spiegazione: La sintassi `throw new ...` serve materialmente a sollevare un'eccezione interrompendo il normale flusso. È un'operazione del tutto lecita all'interno di un blocco `catch` per rilanciare l'errore o sollevarne uno di tipo diverso.

- **Domanda 7**

Testo: Date due classi `A` e `B`, la seguente inizializzazione è corretta:

```
ArrayList<A> a = new ArrayList<B>();
```

Risposta esatta: Falso

Spiegazione: I Generics indicano una struttura (come una lista) che può contenere rigorosamente solo oggetti di un determinato tipo o delle sue sottoclassi. Tuttavia, in Java, i Generics sono invarianti: un `ArrayList<B>` non è considerato compatibile con un `ArrayList<A>`, anche se `B` fosse figlio di `A`.

- **Domanda 8**

Testo: Siano dati i seguenti oggetti: `o` di tipo `Comparator`, `i` di tipo `ArrayList`. Dopo aver fatto `i.sort(o)`; l'oggetto `o` diventa `null`.

Risposta esatta: Falso

Spiegazione: Le variabili che rappresentano oggetti o array vengono allocate nello Stack e contengono un riferimento che punta alla memoria Heap. Quando questo oggetto viene passato a un metodo, ciò che viene copiato è il suo riferimento. Il metodo `sort` utilizzerà il riferimento ma non ha il potere di annullare la variabile originale nello Stack.

- **Domanda 9**

Testo: Le istruzioni `Object[] x = new Object[10]; x[0] = "elemento";` sono illegali in Java, perché per gli array non è supportato il polimorfismo; questo è il motivo principale per l'esistenza del Java Collection Framework.

Risposta esatta: Falso

Spiegazione: In Java, gli array supportano pienamente il polimorfismo di sottotipo. Essendo `String` ("elemento") una sottoclasse di `Object`, l'assegnazione è del tutto lecita. Il Collection Framework non serve per sopperire a una mancanza di polimorfismo negli array, ma per offrire strutture dati ridimensionabili dinamicamente (come le Liste) e metodi di utilità avanzati.

- **Domanda 10**

Testo: Una classe Java definita come `abstract` può essere usata all'interno di gerarchie di classi con ereditarietà multipla.

Risposta esatta: Falso

Spiegazione: Java non supporta in alcun caso l'ereditarietà multipla di stato per le classi, indipendentemente dal fatto che siano `abstract` o concrete. Una classe può estendere al massimo **una sola** altra classe (per evitare il *Diamond Problem*). L'ereditarietà multipla di tipo è permessa esclusivamente implementando molteplici `interface`.

- **Domanda 11**

Testo: Siano dati due oggetti `a` e `b`, per i quali il test `a.hashCode() == b.hashCode()` ritorna `true`. In questo caso, si può stabilire con certezza che `a` e `b` sono identici.

Risposta esatta: Falso

Spiegazione: Il contratto formale di Java garantisce che due oggetti uguali (secondo il metodo `equals()`) abbiano lo stesso hash, ma **non garantisce il viceversa**. Poiché l'hash è un numero intero limitato a 32 bit, due oggetti completamente diversi possono generare matematicamente lo stesso codice (fenomeno noto come *Collisione Hash*). Pertanto, l'uguaglianza dell'hash non implica l'identità né l'uguaglianza logica.

- **Domanda 12**

Testo: In una classe possono coesistere più metodi con lo stesso nome ma firme (numero e tipo dei parametri) diverse.

Risposta esatta: Vero

Spiegazione: Questa è l'esatta definizione del polimorfismo ad-hoc noto come *Overloading* (sovraccarico). Il compilatore riesce a distinguere quale metodo invocare a runtime analizzando il tipo statico e il numero degli argomenti passati.

- **Domanda 13**

Testo: La parola chiave `finally` serve a terminare l'esecuzione del programma.

Risposta esatta: Falso

Spiegazione: Il blocco `finally` viene utilizzato nei costrutti `try-catch` per definire una porzione di codice che verrà **sempre** eseguita (es. chiudere una connessione a un database o un file), sia che un'eccezione venga sollevata, sia che tutto vada a buon fine. Per forzare la terminazione del programma si usa invece `System.exit()`.

- **Domanda 14**

Testo: Si consideri un attributo `x` dichiarato come `protected` nella classe `C` del package `P1`; `x` non è visibile da una classe `D` appartenente a un package `P2`, a meno che `D` non erediti da `C`.

Risposta esatta: Vero

Spiegazione: Il modificatore `protected` rende l'attributo visibile a tutte le classi all'interno dello *stesso package* (`P1`) e a tutte le *sottoclassi*, indipendentemente dal package in cui si trovano. Poiché `D` si trova in un package diverso (`P2`), l'unico modo per accedere a `x` è che `D` sia una sottoclasse (figlia) di `C`.

- **Domanda 15**

Testo: Il garbage collector è una funzionalità del supporto run-time di Java molto utile, in quanto gestisce in maniera automatica la deallocazione di oggetti non più utilizzati dal programma, semplificando il lavoro del programmatore ed evitandone potenziali errori.

Risposta esatta: Vero

Spiegazione: A differenza del C/C++, in cui lo sviluppatore deve allocare e liberare manualmente la memoria (rischiando i temuti *memory leak*), la JVM utilizza il Garbage Collector per rintracciare e distruggere in automatico gli oggetti allocati nella *Heap* che non sono più raggiungibili da alcun riferimento attivo nello *Stack*.

- **Domanda 16**

Testo: Quando in Java si parla di un metodo *overloaded* si fa riferimento a un metodo "sovraccarico", cioè la cui implementazione genera una computazione particolarmente pesante.

Risposta esatta: Falso

Spiegazione: Il termine "sovraccarico" in Java non ha alcuna correlazione con le prestazioni, la memoria o la complessità computazionale dell'algoritmo. Si riferisce esclusivamente alla pratica architetturale di definire più metodi con lo stesso nome ma parametri differenti all'interno della medesima classe.

- **Domanda 17**

Testo: Un tipo generico è una classe che contiene attributi di tipo `Object` nella propria definizione.

Risposta esatta: Falso

Spiegazione: Prima dell'introduzione dei Generics (Java 5), si usava `Object` per creare classi contenitore generiche, con il rischio di errori a runtime dovuti ai cast. Un tipo generico moderno (es. `class Box<T>`) usa parametri di tipo (T) valutati a tempo di compilazione, garantendo la *type-safety* senza dover dichiarare attributi direttamente come `Object`.

- **Domanda 18**

Testo: Nella gestione delle eccezioni in Java, un blocco `try` deve essere sempre seguito da almeno un blocco `catch`.

Risposta esatta: Falso

Spiegazione: Un blocco `try` non richiede obbligatoriamente un `catch`, purché sia seguito da un blocco `finally` (es. `try {...} finally {...}`). Inoltre, con il costrutto *try-with-resources* introdotto in Java 7, è possibile avere un `try` autonomo senza né `catch` né `finally`, poiché la chiusura delle risorse è gestita in automatico.

- **Domanda 19**

Testo: Siano dati i tipi riferimento A e B, e i tipi generici `C<A>` e `C<B>`. Se B è una sottoclasse di A, allora `C<B>` è una sottoclasse di `C<A>`.

Risposta esatta: Falso

Spiegazione: In Java, i tipi generici sono **invarianti**. Anche se `String` è una sottoclasse di `Object`, una `List<String>` NON è una sottoclasse di `List<Object>`. Se lo fosse, potremmo inserire un numero in una lista di stringhe tramite un riferimento alla superclasse, corrompendo la *type-safety*.

- **Domanda 20**

Testo: In Java, la parola chiave `final` può essere impiegata nella testata del metodo di una classe per impedirne la ridefinizione in una sottoclasse.

Risposta esatta: Vero

Spiegazione: Dichiarare un metodo come `final` "blinda" la sua implementazione. Le sottoclassi erediteranno regolarmente il metodo ma il compilatore bloccherà qualsiasi tentativo di farne l'*overriding*. (Questo forza anche il *Static Binding* per quel metodo).

- **Domanda 21**

Testo: La coercizione di tipo (cast) presente in altri linguaggi quali il C è vietata in Java, in quanto potenziale fonte di errori. Pertanto, un'istruzione `A a = (B) b;` dà sempre un errore in compilazione, indipendentemente dalla definizione dei tipi A e B.

Risposta esatta: Falso

Spiegazione: In Java il casting è assolutamente consentito. Il compilatore blocca il cast solo se i tipi sono su due rami gerarchici completamente non correlati (es. castare un `Integer` in `String`). Se B estende A, l'upcasting è lecito (ed è implicito). Se A estende B, il downcasting è permesso sintatticamente e verrà verificato dalla JVM a runtime (sollevando eventualmente una `ClassCastException`).

- **Domanda 22**

Testo: Quando una classe C implementa un'interfaccia I, deve fornire (o ereditare) l'implementazione di tutti i metodi di I, oppure dichiarare C come **abstract**; in caso contrario, viene generato un errore in compilazione.

Risposta esatta: Vero

Spiegazione: Implementare un'interfaccia equivale a siglare un "contratto". Una classe concreta è obbligata a fornire il corpo per tutti i metodi astratti definiti nell'interfaccia. Se non è in grado di fornirli tutti, deve delegare la responsabilità ai suoi futuri figli dichiarandosi essa stessa come **abstract**.

- **Domanda 23**

Testo: Il tipo statico di una variabile p è quello ad esso associato all'atto della dichiarazione, es., `Persona p = ...`. Il tipo dinamico di p, potenzialmente diverso da quello statico, è invece quello associato all'oggetto a cui p punta in un certo punto dell'esecuzione del programma.

Risposta esatta: Vero

Spiegazione: È l'esatta definizione di tipo statico (dichiarato dal programmatore e controllato dal compilatore) e tipo dinamico (l'effettivo oggetto allocato in memoria con `new` nella fase di runtime). Grazie al polimorfismo, il tipo dinamico può essere una qualsiasi sottoclasse del tipo statico.

- **Domanda 24**

Testo: In Java non esiste ereditarietà multipla fra interfacce: un tipo interfaccia può estendere da un solo altro tipo interfaccia.

Risposta esatta: Falso

Spiegazione: Sebbene l'ereditarietà multipla di stato per le classi sia vietata (una classe usa `extends` verso un solo padre), le interfacce **possono** estendere molteplici interfacce contemporaneamente usando una sintassi separata da virgole (es. `interface K extends I, J { }`), permettendo così l'ereditarietà multipla di tipo.

26 Codici d'esame

26.1 Output di Codice

- **Domanda 25**

Testo: Scrivere l'output del codice in figura:

```
public class Test {
    public static void main(String[] args) {
        A a1 = new A(5);
        A a2 = new A(3);
        A a3 = new A(5);
        System.out.println(!a1.equals(a2) + " " + !a1.equals(a3) );
    }
}

class A {
    int x = 0;
    A(int x) { this.x = x; }
    public boolean equals (Object o){
        return this.x == ((A) o).x;
    }
}
```

Spazi vuoti: []

Risposta esatta: true false

Spiegazione: Il metodo `equals()` di default della classe `Object` verifica l'identità (indirizzi di memoria uguali), ma qui è stato fatto un override per fornire un'uguaglianza logica basata sul campo `x`. Quindi `a1` (`x=5`) è logicamente uguale ad `a3` (`x=5`), ma non ad `a2` (`x=3`). L'operatore di negazione `!` inverte i risultati, stampando `true` (per `!a1.equals(a2)`) e poi `false` (per `!a1.equals(a3)`).

- **Domanda 26**

Testo: Scrivere l'output del codice in figura:

```
import java.util.*;
public class Test {
    public static void main(String[] args) {
        String[] a = {"glass", "grass", "dirt",};
        Set<Integer> s = new HashSet<>();
        List<Integer> l = new ArrayList<>();
        for(String i: a) {
            s.add(i.length());
            l.add(i.length());
        }
        System.out.println(s.size() + l.size());
    }
}
```

Spazi vuoti: []

Risposta esatta: 5

Spiegazione: Una `List` in Java gestisce una collezione dinamica e conserva tutti gli elementi aggiunti, anche i duplicati, quindi la sua dimensione finale (`l.size()`) sarà 3. Un `HashSet`, invece, utilizza `hashCode()` ed `equals()` per non ammettere duplicati. Le lunghezze delle stringhe sono 5 ("glass"), 5 ("grass") e 4 ("dirt"). Poiché il Set scarta il 5 duplicato, la sua dimensione (`s.size()`) sarà 2. La somma delle dimensioni (2 + 3) dà 5.

- **Domanda 27**

Testo: Scrivere l'output del codice in figura:

```
import java.util.*;

public class Test {
    public static void main(String[] args) {
        Group<A> g = new Group<>();
        g.add(new A());
        g.add(new A());
        g.add(new A());
        g.show();
    }
}

class Group<T> {
    List<T> l = new ArrayList<>();
    void add(T obj) { l.add(obj); }
    void show() {
        for (T t : l)
            System.out.println(t);
    }
}

class A {
    static int counter = 20;
    int x;
    A() { x = ++counter; }
    public String toString() { return "" + x; }
}
```

Spazi vuoti: []

Risposta esatta:

21

22

23

Spiegazione: La variabile `counter` è marcata come `static`, il che significa che appartiene alla classe e ne esiste una sola copia globale condivisa per tutta l'esecuzione. Ogni volta che si istanzia un nuovo oggetto chiamando `new A()`, il costruttore

incrementa lo stesso contatore, portandolo progressivamente a 21, 22 e 23, assegnando il valore corrente alla variabile d'istanza `x` di ciascun oggetto.

- **Domanda 28**

Testo: Scrivere l'output del codice in figura:

```
public class Test {
    public static void main(String[] args) {
        B b1 = new B(new A());
        b1.m(10);
        B b2 = new B(new A());
        b2.m(8);
        System.out.println(b2);
    }
}

class A {
    static int x = 15;
    void m(int x) { this.x += x; }
    public String toString() { return "x="+x; }
}

class B {
    A a = null;
    B(A a) { this.a = a; }
    void m(int x) { a.m(x); }
    public String toString() { return a.toString(); }
}
```

Spazi vuoti: []

Risposta esatta: x=33

Spiegazione: Anche qui la variabile `x` nella classe `A` è **static** (globale e condivisa). Il suo valore di partenza è 15. Eseguendo `b1.m(10)`, si va ad aggiungere 10 al valore statico, portandolo a 25. Successivamente, `b2.m(8)` aggiunge 8 allo stesso valore condiviso, che diventa 33. Quando si stampa `b2`, questo invoca a sua volta il `toString()` di `a`, che stamperà l'ultimo valore aggiornato di `x`.

- **Domanda 29**

Testo: Scrivere l'output del codice in figura:

```
public class Test {
    public static void main(String[] args) {
        A obj = new B();
        obj.m(new D());
    }
}
class A {
    void m(C c) { System.out.println("x"); }
}
class B extends A {
    void m(C c) { System.out.println("y"); }
    void m(D c) { System.out.println("z"); }
}
class C {}
class D extends C {}
```

Spazi vuoti: []

Risposta esatta: y

Spiegazione: È il classico "Tranello del Dispatch". Il compilatore valuta i tipi statici: `obj` è di tipo `A`, l'argomento `new D()` è di tipo `D`. Cerca `m(D)` dentro `A`, ma non lo trova. Trova però `m(C)`, che è compatibile (poiché `D` estende `C`). La firma scelta a tempo di compilazione (static binding) è quindi `m(C)`. A runtime, la JVM valuta il tipo dinamico di `obj`, che è `B`. Cerca quindi l'override di `m(C)` dentro `B` e lo esegue, stampando `y`.

- **Domanda 30**

Testo: Scrivere l'output del codice in figura:

```
import java.util.*;
public class Test {
    public static void main(String[] args) {
        A[] a = new A[3];
        for (int i=0; i<a.length; i++) {
            if (i%2 != 0) a[i] = new A(i);
            else a[i] = new B(i);
        }
        ArrayList<A> l = new ArrayList<>(Arrays.asList(a));
        for (A e: l) System.out.print(e.m(2) + " ");
    }
}
class A {
    int x;
    A(int x) { this.x = x+1; }
    public int m(int z) { return x + z; }
}
class B extends A {
    B(int x) { super(x); }
    public int m(int z) { return super.m(z) * 3; }
}
```

Spazi vuoti: []

Risposta esatta: 9 4 15

Spiegazione: Il ciclo for crea oggetti B(0), A(1) e B(2). Il costruttore di A inizializza sempre $x = \text{parametro} + 1$. Quindi gli x interni saranno rispettivamente 1, 2 e 3. Durante l'iterazione sulla lista, viene chiamato `e.m(2)` polimorficamente: - Elemento 0 (tipo B, x=1): Chiama `B.m(2) → super.m(2) * 3 → (1 + 2) * 3 = 9`. - Elemento 1 (tipo A, x=2): Chiama `A.m(2) → 2 + 2 = 4`. - Elemento 2 (tipo B, x=3): Chiama `B.m(2) → super.m(2) * 3 → (3 + 2) * 3 = 15`.

- **Domanda 31**

Testo: Scrivere l'output del codice in figura:

```
import java.util.*;
public class Test {
    public static void main(String[] args) {
        List<A> ls = new ArrayList<>();
        ls.add(new A("Pordenone"));
        ls.add(new A("Trento"));
        ls.add(new A("Venezia"));
        Collections.sort(ls);
        for(A a: ls) System.out.println(a.s.length());
    }
}
class A implements Comparable<A> {
    String s;
    A(String s) { this.s = s; }
    public int compareTo(A a) { return (s.length() - a.s.length()); }
}
```

Spazi vuoti: []

Risposta esatta:

6

7

9

Spiegazione: La classe **A** implementa l'interfaccia **Comparable<A>** sovrascrivendo il metodo **compareTo**. La logica di confronto sottrae le lunghezze delle due stringhe, imponendo un ordinamento naturale crescente basato esclusivamente sulla lunghezza della stringa (non in ordine alfabetico). Le lunghezze originali sono 9 ("Pordenone"), 6 ("Trento") e 7 ("Venezia"). **Collections.sort** riordina la lista, che diventerà Trento, Venezia, Pordenone. Il ciclo **for** stampa le rispettive lunghezze.

- **Domanda 32**

Testo: Scrivere l'output del codice in figura:

```
public class Test {
    static String removeLast(String str) {
        return str.substring(0, str.length() - 1);
    }
    public static void main(String[] args) {
        A obj1 = new A("Java");
        A obj2 = new B("Python");
        B obj3 = new B("Ruby");
        I obj4 = new B("Scala");
        System.out.println(obj1.m()+" " + obj2.m()+" " + obj3.m() + " " + obj4.m());
    }
}
interface I { public String m(); }
class A {
    String s;
    A(String s) { this.s = s; }
    public String m() { return s; }
}
class B extends A implements I {
    B(String s) { super(s); }
    public String m() { return Test.removeLast(s); }
}
```

Spazi vuoti: []

Risposta esatta: Java Pytho Rub Scal

Spiegazione: Indipendentemente dai tipi statici con cui sono state dichiarate le variabili (A, B o l'interfaccia I), a runtime agisce il polimorfismo di *Dynamic Binding*. Solamente obj1 è istanziato come oggetto reale A, quindi restituisce la stringa intatta ("Java"). Le variabili obj2, obj3 e obj4 sono state tutte istanziate come nuovi oggetti B. La JVM eseguirà per tutte l'override di m() definito in B, che tronca l'ultimo carattere della parola.

26.2 Identificazione Errori

- **Domanda 33**

Testo: Per il codice sottostante, indicare il tipo di errore, la linea e la motivazione:

```
public class Test {  
    public static void main(String[] args) {  
        AAA a = new AAA();  
        BBB b = a.m(5);  
        b.m();  
    }  
}  
class AAA {  
    BBB m(int x) { return new BBB(x); }  
}  
class BBB {  
    int x;  
    BBB(int x) throws MyException { this.x = x; throw new MyException(); }  
    void m() { }  
}  
class MyException extends Exception {}
```

Spazi vuoti: Tipo errore [menu a tendina] - Linea [] - Motivazione []

Risposta esatta: Errore a compilazione | Linea 9 | Eccezione Checked non gestita

Spiegazione: `MyException` estende direttamente `Exception`, per cui rientra nelle Checked Exceptions, le quali obbligano il programmatore a gestirle a tempo di compilazione. Il costruttore di `BBB` dichiara (`throws`) questa eccezione. Alla linea 9, la chiamata a `new BBB(x)` rischia di sollevarla, ma non viene né catturata da un blocco `try-catch`, né propagata aggiungendo un `throws` alla firma del metodo `m()`. Il compilatore blocca l'operazione.

- **Domanda 34**

Testo: Per il codice sottostante, indicare il tipo di errore, la linea e la motivazione:

```
public class Test {
    static final int MAX = 10;
    public static void main(String[] args) {
        A a = new A(10);
        A a1 = new C();
        A a2 = new B();
        C c1 = (C) a1;
        C c2 = (C) a2;
        a.m("dark");
        c1.m("light");
        c2.m("what");
    }
}
class A {
    A(int x) { }
    public int m(String s) {
        return s.length();
    }
}
class B extends A {
    B() { super(0); }
}
class C extends B {
    C() { super(); }
}
```

Spazi vuoti: Tipo errore [menu a tendina] - Linea [] - Motivazione []

Risposta esatta: Errore a runtime | Linea 8 | ClassCastException

Spiegazione: Alla linea 6, ad **a2** viene assegnato in memoria (tipo dinamico) un oggetto di tipo B. Alla linea 8 si tenta un downcasting (**C c2 = (C) a2;**), forzando un oggetto che è materialmente un B a diventare un C (la sua sottoclasse). Poiché l'oggetto effettivo nella memoria Heap non è compatibile con quel tipo più specifico, la JVM andrà in crash durante l'esecuzione sollevando una **ClassCastException**.

- **Domanda 35**

Testo: Per il codice sottostante, indicare il tipo di errore, la linea e la motivazione:

```
public class Test {
    public static void main(String[] args) {
        final A a = new A();
        A a1 = new A();
        a1.doSomething();
    }
}
class A {
    int x;
    A(int x) { this.x = x; }
    A(int x, int y){ this.x = y; }
    void doSomething() {
        System.out.println("No_way");
    }
}
```

Spazi vuoti: Tipo errore [menu a tendina] - Linea [] - Motivazione []

Risposta esatta: Errore a compilazione | Linea 3 | Costruttore base non trovato (mancante)

Spiegazione: In Java, il costruttore di default (senza parametri) viene fornito automaticamente solo se il programmatore non ha scritto alcun costruttore esplicito. Avendo la classe A due costruttori personalizzati (che richiedono `int x` o `int x, int y`), il costruttore `A()` sparisce. L'invocazione `new A()` alle linee 3 e 4 non trova corrispondenze valide, generando un errore al momento della compilazione (l'errore fatale scatta per primo alla linea 3).

- **Domanda 36**

Testo: Per il codice sottostante, indicare il tipo di errore, la linea e la motivazione:

```
public class Test {
    public static void main(String[] args) {
        A a = new A();
        System.out.print(a.equals(a));
    }
}
class A {
    private B b;
    public String toString() { return b.equals(this); }
}
class B {
    private int x;
}
```

Spazi vuoti: Tipo errore [menu a tendina] - Linea [] - Motivazione []
Risposta esatta: Errore a compilazione | Linea 9 | Tipo di ritorno incompatibile
Spiegazione: Il metodo `toString()` ereditato da `Object` esige come tipo di ritorno obbligatorio una `String`. Nella riga 9 viene effettuato un override in cui si tenta di restituire `b.equals(this)`, che è un'espressione logica che valuta e restituisce un tipo primitivo `boolean`. Poiché un booleano non può essere convertito implicitamente in `String`, il compilatore segnala l'incompatibilità dei tipi.

- **Domanda 37**

Testo: Per il codice sottostante, indicare il tipo di errore, la linea e la motivazione:

```
public class Test {  
    public static void main(String[] args) {  
        B x = new B(); x.m();  
        B y = new B(3); y.m();  
        A z = new B(3); z.m();  
    }  
}  
  
class A {  
    int x = 2;  
    A() { System.out.println("A"); }  
    void m() { System.out.println("A" + x); }  
}  
  
class B extends A {  
    B() { System.out.println("B"); }  
    B(int x) { this.x = x; }  
    void m() { System.out.println("B" + x); }  
}
```

Spazi vuoti: Tipo errore [menu a tendina] - Linea [] - Motivazione []
Risposta esatta: Nessun errore (Il codice è corretto) | Linea n.d. | -
Spiegazione: Il codice è sintatticamente corretto. La classe `B` estende `A`. Quando si chiama `new B(3)`, viene invocato implicitamente il costruttore `A()` della superclasse. Il campo `x` ereditato viene poi sovrascritto dal valore passato nel costruttore di `B`. Non ci sono violazioni di visibilità o di tipi.

- **Domanda 38**

Testo: Per il codice sottostante, indicare il tipo di errore, la linea e la motivazione:

```
public class Test {
    public static void main(String[] args) {
        C obj = new C();
        int a = C.m1();
        int b = obj.m3();
    }
}
class C {
    static int x = 5;
    int y = 3;
    static public int m1() { return y + m2(); }
    static private int m2() { return x; }
    public int m3() { return m1(); }
}
```

Spazi vuoti: Tipo errore [menu a tendina] - Linea [] - Motivazione []

Risposta esatta: Errore a compilazione | Linea 10 | Accesso a variabile d'istanza da contesto statico

Spiegazione: Il metodo `m1()` è dichiarato **static**. In Java, un metodo statico appartiene alla classe e non può accedere direttamente a variabili d'istanza (non statiche) come `y`, poiché tali variabili esistono solo quando viene istanziato un oggetto specifico. L'accesso a `y` dentro `m1()` causa un errore di compilazione.

- **Domanda 39**

Testo: Per il codice sottostante, indicare il tipo di errore, la linea e la motivazione:

```
public class Test {
    public static void main(String[] args) {
        A a = new B();
        a.m(new D());
    }
}
class A {
    void m(C c) { System.out.println("x"); }
}
class B extends A {
    void m(C c) { System.out.println("y"); }
    void m(D c) { System.out.println("z"); }
}
class C {}
class D extends C {}
```

Spazi vuoti: Tipo errore [menu a tendina] - Linea [] - Motivazione []

Risposta esatta: Nessun errore (Il codice è corretto) | Linea n.d. | -

Spiegazione: Questo è un esercizio di polimorfismo avanzato. Il compilatore sceglie la firma `m(C)` perché il tipo statico del chiamante è `A` (che conosce solo `m(C)`). A runtime, la JVM usa il tipo dinamico (`B`) per eseguire l'override di `m(C)`, stampando `y`. Se il tipo statico fosse stato `B`, il compilatore avrebbe scelto `m(D)` (firma più specifica), stampando `z`. Il codice è valido in entrambi i casi.